

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent	12418565
Kind Code	B2
Date of Patent	September 16, 2025
Inventor(s)	Vissamsetty; Venu et al.

Deceiving attackers accessing network data

Abstract

Endpoints in a network execute a sensor module that intercepts responses from a directory service to a source on the endpoint. The sensor module determines if the source is sanctioned, such as by referencing a sanctioned list of applications. If the source is not sanctioned, then a simulated response can be provided to the source that references a decoy server.

Inventors: Vissamsetty; Venu (San Jose, CA), Gupta; Anil (Bangalore, IN), Ramchetty; Harinath Vishwanath (Bangalore, IN)

Applicant: SENTINELONE, INC. (Mountain View, CA)

Family ID: 1000008818103

Assignee: SentinelOne, Inc. (Mountain View, CA)

Appl. No.: 18/346148

Filed: June 30, 2023

Prior Publication Data

Document Identifier	Publication Date
US 20240007503 A1	Jan. 04, 2024

Related U.S. Application Data

continuation parent-doc US 16849813 20200415 US 11695800 child-doc US 18346148
continuation-in-part parent-doc US 16543189 20190816 US 11616812 20230328 child-doc US 16849813
continuation-in-part parent-doc US 15383522 20161219 US 10599842 20200324 child-doc US 16543189

Publication Classification

Int. Cl.: **H04L9/40** (20220101); **G06F21/55** (20130101); **G06F21/56** (20130101); **H04L61/4505** (20220101); **H04L61/5007** (20220101); H04L61/4523 (20220101)

U.S. Cl.:

CPC **H04L63/1491** (20130101); **G06F21/554** (20130101); **G06F21/566** (20130101); **H04L61/4505** (20220501); **H04L61/5007** (20220501); **H04L63/1416** (20130101); G06F2221/2123 (20130101); H04L61/4523 (20220501)

Field of Classification Search

CPC: H04L (63/1491); H04L (41/12); H04L (61/4505); H04L (61/5007); H04L (61/4523); H04L (63/1416); G06F (21/566); G06F (21/554); G06F (2221/2123)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
4979118	12/1989	Kheradpir	N/A	N/A
5311593	12/1993	Carmi	N/A	N/A
5754938	12/1997	Herz et al.	N/A	N/A
6026474	12/1999	Carter et al.	N/A	N/A
6154844	12/1999	Touboul et al.	N/A	N/A
6157953	12/1999	Chang et al.	N/A	N/A
6728716	12/2003	Bhattacharya et al.	N/A	N/A
6804780	12/2003	Touboul	N/A	N/A
6836888	12/2003	Basu et al.	N/A	N/A
7076696	12/2005	Stringer	N/A	N/A
7093239	12/2005	Van der Made	N/A	N/A
7181769	12/2006	Keanini et al.	N/A	N/A
7225468	12/2006	Waisman et al.	N/A	N/A
7299294	12/2006	Bruck et al.	N/A	N/A
7305546	12/2006	Miller	N/A	N/A
7322044	12/2007	Hrastar	N/A	N/A
7464407	12/2007	Nakae et al.	N/A	N/A
7530106	12/2008	Zaitsev et al.	N/A	N/A
7543269	12/2008	Krueger	N/A	N/A
7546587	12/2008	Marr et al.	N/A	N/A
7574740	12/2008	Kennis	N/A	N/A
7596807	12/2008	Ptacek et al.	N/A	N/A
7596808	12/2008	Wilkinson et al.	N/A	N/A
7665139	12/2009	Szor et al.	N/A	N/A
7710933	12/2009	Sundaralingam et al.	N/A	N/A
7739516	12/2009	Bender et al.	N/A	N/A
7832012	12/2009	Huddleston	N/A	N/A
7882538	12/2010	Palmer	N/A	N/A
7890612	12/2010	Todd et al.	N/A	N/A

7937755	12/2010	Guruswamy	N/A	N/A
7958549	12/2010	Nakae et al.	N/A	N/A
7975018	12/2010	Unrau et al.	N/A	N/A
7984129	12/2010	Vaught	N/A	N/A
8015605	12/2010	Yegneswaran	N/A	N/A
8024795	12/2010	Newton	N/A	N/A
8042186	12/2010	Polyakov et al.	N/A	N/A
8056134	12/2010	Ogilvie	713/188	G06F 21/566
8065722	12/2010	Barford et al.	N/A	N/A
8078556	12/2010	Adi et al.	N/A	N/A
8082471	12/2010	Khan	N/A	N/A
8131281	12/2011	Hildner et al.	N/A	N/A
8141154	12/2011	Gruzman et al.	N/A	N/A
8156556	12/2011	Krishnamurthy	N/A	N/A
8171545	12/2011	Cooley et al.	N/A	N/A
8181033	12/2011	Paul et al.	N/A	N/A
8181250	12/2011	Rafalovich et al.	N/A	N/A
8204984	12/2011	Aziz	N/A	N/A
8205035	12/2011	Reddy et al.	N/A	N/A
8230505	12/2011	Ahrens et al.	N/A	N/A
8239947	12/2011	Glick et al.	N/A	N/A
8296842	12/2011	Singh et al.	N/A	N/A
8321940	12/2011	Pereira et al.	N/A	N/A
8327442	12/2011	Herz et al.	N/A	N/A
8353033	12/2012	Chen et al.	N/A	N/A
8370931	12/2012	Chien et al.	N/A	N/A
8375444	12/2012	Aziz et al.	N/A	N/A
8375447	12/2012	Amoroso et al.	N/A	N/A
8413238	12/2012	Sutton	N/A	N/A
8413241	12/2012	Weeks et al.	N/A	N/A
8418250	12/2012	Morris et al.	N/A	N/A
8438386	12/2012	Hegli et al.	N/A	N/A
8438626	12/2012	Anderson et al.	N/A	N/A
8443442	12/2012	Wang et al.	N/A	N/A
8474044	12/2012	Zawadowskiy et al.	N/A	N/A
8488466	12/2012	Breslin et al.	N/A	N/A
8528057	12/2012	Garrett	N/A	N/A
8528087	12/2012	Hsu et al.	N/A	N/A
8538578	12/2012	Battles et al.	N/A	N/A
8539578	12/2012	Zhou et al.	N/A	N/A
8539582	12/2012	Aziz et al.	N/A	N/A
8549643	12/2012	Shou	N/A	N/A
8555385	12/2012	Bhatkar et al.	N/A	N/A
8561177	12/2012	Aziz et al.	N/A	N/A
8566946	12/2012	Aziz et al.	N/A	N/A
8607054	12/2012	Ramarathinam et al.	N/A	N/A
8607340	12/2012	Wright	N/A	N/A
8627475	12/2013	Loveland et al.	N/A	N/A
8677494	12/2013	Edery et al.	N/A	N/A

8713306	12/2013	Bennett	N/A	N/A
8719937	12/2013	Sundaram et al.	N/A	N/A
8725898	12/2013	Vincent	N/A	N/A
8726389	12/2013	Morris et al.	N/A	N/A
8732296	12/2013	Thomas et al.	N/A	N/A
8752173	12/2013	Yadav	N/A	N/A
8789135	12/2013	Pani	N/A	N/A
8793151	12/2013	Delzoppo et al.	N/A	N/A
8806638	12/2013	Mani	N/A	N/A
8821242	12/2013	Hinman et al.	N/A	N/A
8839369	12/2013	Dai et al.	N/A	N/A
8844041	12/2013	Kienzle et al.	N/A	N/A
8849880	12/2013	Thelen	N/A	N/A
8850582	12/2013	Endoh et al.	N/A	N/A
8880435	12/2013	Catlett et al.	N/A	N/A
8881282	12/2013	Aziz et al.	N/A	N/A
8893278	12/2013	Chechik	N/A	N/A
8898788	12/2013	Aziz et al.	N/A	N/A
8904527	12/2013	Dawson et al.	N/A	N/A
8943594	12/2014	Arrowood	N/A	N/A
8949986	12/2014	Ben-Shalom	N/A	N/A
8959338	12/2014	Snow et al.	N/A	N/A
8973142	12/2014	Shulman et al.	N/A	N/A
8984637	12/2014	Karecha et al.	N/A	N/A
9009829	12/2014	Stolfo et al.	N/A	N/A
9027135	12/2014	Aziz	N/A	N/A
9043920	12/2014	Gula et al.	N/A	N/A
9081747	12/2014	Tabieros et al.	N/A	N/A
9117078	12/2014	Chien et al.	N/A	N/A
9141792	12/2014	Baluda et al.	N/A	N/A
9166993	12/2014	Liu	N/A	N/A
9185136	12/2014	Dulkin et al.	N/A	N/A
9195480	12/2014	Wang et al.	N/A	N/A
9197601	12/2014	Pasdar	N/A	N/A
9213838	12/2014	Lu	N/A	N/A
9225734	12/2014	Hastings	N/A	N/A
9240976	12/2015	Murchison	N/A	N/A
9246774	12/2015	Mataitis et al.	N/A	N/A
9270690	12/2015	Kraitsman et al.	N/A	N/A
9305165	12/2015	Snow et al.	N/A	N/A
9329973	12/2015	Bhuyan	N/A	N/A
9330259	12/2015	Klein et al.	N/A	N/A
9356942	12/2015	Joffe	N/A	N/A
9356950	12/2015	Vissamsetty et al.	N/A	N/A
9369476	12/2015	Chekina et al.	N/A	N/A
9386034	12/2015	Cochenour	N/A	N/A
9398001	12/2015	Tidd	N/A	N/A
9407602	12/2015	Feghali et al.	N/A	N/A
9413721	12/2015	Morris et al.	N/A	N/A
9420002	12/2015	McGovern et al.	N/A	N/A

9430646	12/2015	Mushtaq et al.	N/A	N/A
9432360	12/2015	Triandopoulos et al.	N/A	N/A
9438614	12/2015	Herz	N/A	N/A
9495188	12/2015	Ettema et al.	N/A	N/A
9503470	12/2015	Gertner et al.	N/A	N/A
9547516	12/2016	Thakkar et al.	N/A	N/A
9578045	12/2016	Jaroch et al.	N/A	N/A
9591006	12/2016	Siva et al.	N/A	N/A
9601000	12/2016	Gruss et al.	N/A	N/A
9602531	12/2016	Wallace et al.	N/A	N/A
9606893	12/2016	Gupta et al.	N/A	N/A
9607146	12/2016	Sridhara et al.	N/A	N/A
9609019	12/2016	Vissamsetty et al.	N/A	N/A
9628498	12/2016	Aziz et al.	N/A	N/A
9641544	12/2016	Treat et al.	N/A	N/A
9641550	12/2016	Kraitsman et al.	N/A	N/A
9661023	12/2016	Fang et al.	N/A	N/A
9705904	12/2016	Davis et al.	N/A	N/A
9710648	12/2016	Weingarten	N/A	N/A
9712547	12/2016	Touboul et al.	N/A	N/A
9749349	12/2016	Czarny et al.	N/A	N/A
9769204	12/2016	Vissamsetty et al.	N/A	N/A
9772832	12/2016	Rubio	N/A	N/A
9781148	12/2016	Mahaffey et al.	N/A	N/A
9807092	12/2016	Gutzmann	N/A	N/A
9807115	12/2016	Kolton et al.	N/A	N/A
9813451	12/2016	Honda et al.	N/A	N/A
9871766	12/2017	Syed et al.	N/A	N/A
9877210	12/2017	Hildner et al.	N/A	N/A
9888032	12/2017	Dekel et al.	N/A	N/A
9898763	12/2017	Vaynblat et al.	N/A	N/A
9935968	12/2017	Pal et al.	N/A	N/A
9942270	12/2017	Vissamsetty et al.	N/A	N/A
10025928	12/2017	Jaroch et al.	N/A	N/A
10044675	12/2017	Ettema et al.	N/A	N/A
10102374	12/2017	Cohen et al.	N/A	N/A
10169586	12/2018	Maciejak et al.	N/A	N/A
10193920	12/2018	Satish et al.	N/A	N/A
10237282	12/2018	Nelson et al.	N/A	N/A
10250636	12/2018	Vissamsetty et al.	N/A	N/A
10257224	12/2018	Jaroch et al.	N/A	N/A
10284591	12/2018	Giuliani et al.	N/A	N/A
10375110	12/2018	Vissamsetty et al.	N/A	N/A
10382484	12/2018	Shayevitz et al.	N/A	N/A
10476891	12/2018	Vissamsetty et al.	N/A	N/A
10509905	12/2018	Vissamsetty et al.	N/A	N/A
10542044	12/2019	Vissamsetty et al.	N/A	N/A
10567431	12/2019	Vissamsetty et al.	N/A	N/A
10574698	12/2019	Sharifi	N/A	N/A
10599842	12/2019	Vissamsetty et al.	N/A	N/A

10599844	12/2019	Schmidtler et al.	N/A	N/A
10609074	12/2019	Vissamsetty et al.	N/A	N/A
10757090	12/2019	Kahol et al.	N/A	N/A
10769267	12/2019	Li et al.	N/A	N/A
10826941	12/2019	Jain et al.	N/A	N/A
10855671	12/2019	Kahol et al.	N/A	N/A
10938854	12/2020	Strogov et al.	N/A	N/A
11032301	12/2020	Mandrychenko et al.	N/A	N/A
11038658	12/2020	Vissamsetty et al.	N/A	N/A
11171974	12/2020	Gertner et al.	N/A	N/A
11310262	12/2021	Oliphant et al.	N/A	N/A
11316879	12/2021	Ott et al.	N/A	N/A
11330005	12/2021	Beauchesne et al.	N/A	N/A
11379607	12/2021	Swafford	N/A	N/A
11470115	12/2021	Vissamsetty et al.	N/A	N/A
11481503	12/2021	Gitelman et al.	N/A	N/A
11507663	12/2021	Cohen et al.	N/A	N/A
11522894	12/2021	Weingarten et al.	N/A	N/A
11539722	12/2021	Singh et al.	N/A	N/A
11579857	12/2022	Montag et al.	N/A	N/A
11580218	12/2022	Salem et al.	N/A	N/A
11615184	12/2022	Kutt et al.	N/A	N/A
11695800	12/2022	Vissamsetty et al.	N/A	N/A
11734303	12/2022	Cruanes et al.	N/A	N/A
11888897	12/2023	Vissamsetty et al.	N/A	N/A
11997139	12/2023	Mssamsetty et al.	N/A	N/A
2002/0010800	12/2001	Riley et al.	N/A	N/A
2002/0016826	12/2001	Johansson et al.	N/A	N/A
2002/0078382	12/2001	Sheikh et al.	N/A	N/A
2002/0093917	12/2001	Knobbe et al.	N/A	N/A
2002/0095607	12/2001	Lin-Hendel	N/A	N/A
2002/0178374	12/2001	Swimmer et al.	N/A	N/A
2002/0194489	12/2001	Almogly et al.	N/A	N/A
2003/0065950	12/2002	Yarborough	N/A	N/A
2003/0126464	12/2002	McDaniel et al.	N/A	N/A
2003/0145225	12/2002	Bruton, III et al.	N/A	N/A
2003/0145226	12/2002	Bruton, III et al.	N/A	N/A
2003/0152034	12/2002	Zhang et al.	N/A	N/A
2003/0188189	12/2002	Desai et al.	N/A	N/A
2003/0223367	12/2002	Shay et al.	N/A	N/A
2004/0083369	12/2003	Erlingsson et al.	N/A	N/A
2004/0172557	12/2003	Nakae et al.	N/A	N/A
2004/0243699	12/2003	Koclanes et al.	N/A	N/A
2004/0255157	12/2003	Ghanea-Hercock	N/A	N/A
2005/0050353	12/2004	Thiele et al.	N/A	N/A
2005/0076235	12/2004	Ormazabal et al.	N/A	N/A
2005/0076238	12/2004	Ormazabal et al.	N/A	N/A
2005/0108562	12/2004	Khazan et al.	N/A	N/A
2005/0138402	12/2004	Yoon et al.	N/A	N/A
2005/0160480	12/2004	Birt et al.	N/A	N/A

2005/0193173	12/2004	Ring et al.	N/A	N/A
2005/0198507	12/2004	Brender et al.	N/A	N/A
2005/0204157	12/2004	Johnson	N/A	N/A
2005/0223239	12/2004	Dotan	713/188	G06F 21/53
2005/0240906	12/2004	Kinderknecht et al.	N/A	N/A
2005/0240989	12/2004	Kim et al.	N/A	N/A
2005/0288939	12/2004	Peled	705/325	G06Q 10/10
2006/0053490	12/2005	Herz et al.	N/A	N/A
2006/0085543	12/2005	Hrastar	N/A	N/A
2006/0101515	12/2005	Amoroso et al.	N/A	N/A
2006/0126522	12/2005	Oh	N/A	N/A
2006/0161989	12/2005	Reshef et al.	N/A	N/A
2006/0179296	12/2005	Bartlett et al.	N/A	N/A
2006/0203774	12/2005	Carrion-Rodrigo	N/A	N/A
2006/0209701	12/2005	Zhang et al.	N/A	N/A
2006/0230129	12/2005	Swami et al.	N/A	N/A
2006/0236401	12/2005	Fosdick	N/A	N/A
2007/0022090	12/2006	Graham	N/A	N/A
2007/0025374	12/2006	Stefan et al.	N/A	N/A
2007/0067623	12/2006	Ward	N/A	N/A
2007/0097976	12/2006	Wood et al.	N/A	N/A
2007/0100905	12/2006	Masters et al.	N/A	N/A
2007/0101431	12/2006	Clift et al.	N/A	N/A
2007/0115993	12/2006	Cohen	N/A	N/A
2007/0143827	12/2006	Nicodemus et al.	N/A	N/A
2007/0143851	12/2006	Nicodemus et al.	N/A	N/A
2007/0177499	12/2006	Gavrilescu et al.	N/A	N/A
2007/0192500	12/2006	Lum	N/A	N/A
2007/0208822	12/2006	Wang et al.	N/A	N/A
2007/0208936	12/2006	Ramos	N/A	N/A
2007/0209070	12/2006	Yadav	N/A	N/A
2007/0226320	12/2006	Hager et al.	N/A	N/A
2007/0240215	12/2006	Flores et al.	N/A	N/A
2007/0250930	12/2006	Aziz et al.	N/A	N/A
2007/0266422	12/2006	Germano et al.	N/A	N/A
2007/0282782	12/2006	Carey et al.	N/A	N/A
2008/0005782	12/2007	Aziz	N/A	N/A
2008/0018927	12/2007	Martin et al.	N/A	N/A
2008/0022000	12/2007	Furuya et al.	N/A	N/A
2008/0034429	12/2007	Schneider	N/A	N/A
2008/0046989	12/2007	Wahl	N/A	N/A
2008/0060074	12/2007	Okuyama	N/A	N/A
2008/0071728	12/2007	Lim	N/A	N/A
2008/0082722	12/2007	Savagaonkar et al.	N/A	N/A
2008/0083034	12/2007	Kim et al.	N/A	N/A
2008/0098476	12/2007	Syversen	N/A	N/A
2008/0104046	12/2007	Singla et al.	N/A	N/A
2008/0127346	12/2007	Oh et al.	N/A	N/A
2008/0148381	12/2007	Aaron	N/A	N/A
2008/0162397	12/2007	Zaltzman	N/A	N/A

2008/0168559	12/2007	Touitou et al.	N/A	N/A
2008/0170566	12/2007	Akimoto	N/A	N/A
2008/0229415	12/2007	Kapoor et al.	N/A	N/A
2008/0244694	12/2007	Neystadt et al.	N/A	N/A
2008/0267144	12/2007	Jano et al.	N/A	N/A
2009/0077664	12/2008	Hsu et al.	N/A	N/A
2009/0089040	12/2008	Monastyrsky et al.	N/A	N/A
2009/0104046	12/2008	Martin et al.	N/A	N/A
2009/0158407	12/2008	Nicodemus et al.	N/A	N/A
2009/0158418	12/2008	Rao et al.	N/A	N/A
2009/0170566	12/2008	Kwon et al.	N/A	N/A
2009/0199296	12/2008	Xie et al.	N/A	N/A
2009/0241173	12/2008	Troyansky	726/5	G06F 21/6218
2009/0249466	12/2008	Motil et al.	N/A	N/A
2009/0249470	12/2008	Litvin et al.	N/A	N/A
2009/0254973	12/2008	Kwan	N/A	N/A
2009/0288158	12/2008	Izatt et al.	N/A	N/A
2009/0296641	12/2008	Bienas et al.	N/A	N/A
2009/0327688	12/2008	Li et al.	N/A	N/A
2009/0328196	12/2008	Bovee	N/A	N/A
2010/0005339	12/2009	Hooks	N/A	N/A
2010/0077483	12/2009	Stolfo et al.	N/A	N/A
2010/0122317	12/2009	Yadav	N/A	N/A
2010/0122343	12/2009	Ghosh et al.	N/A	N/A
2010/0169973	12/2009	Kim et al.	N/A	N/A
2010/0269175	12/2009	Stolfo et al.	N/A	N/A
2010/0293615	12/2009	Ye	N/A	N/A
2010/0299430	12/2009	Powers et al.	N/A	N/A
2010/0332498	12/2009	Svore et al.	N/A	N/A
2010/0333177	12/2009	Donley et al.	N/A	N/A
2011/0023118	12/2010	Wright et al.	N/A	N/A
2011/0067107	12/2010	Weeks et al.	N/A	N/A
2011/0078309	12/2010	Bloch	N/A	N/A
2011/0099633	12/2010	Aziz	N/A	N/A
2011/0113427	12/2010	Dotan	N/A	N/A
2011/0138456	12/2010	Ormazabal et al.	N/A	N/A
2011/0141937	12/2010	Breslin et al.	N/A	N/A
2011/0145920	12/2010	Mahaffey et al.	N/A	N/A
2011/0167494	12/2010	Bowen	N/A	N/A
2011/0178930	12/2010	Scheidt et al.	N/A	N/A
2011/0185430	12/2010	Sallam	N/A	N/A
2011/0209219	12/2010	Zeitlin et al.	N/A	N/A
2011/0214176	12/2010	Burch et al.	N/A	N/A
2011/0214182	12/2010	Adams et al.	N/A	N/A
2011/0219443	12/2010	Hampel	N/A	N/A
2011/0219449	12/2010	St. Neitzel	N/A	N/A
2011/0225655	12/2010	Niemelä et al.	N/A	N/A
2011/0247071	12/2010	Hooks et al.	N/A	N/A
2011/0252434	12/2010	Stokes	N/A	N/A

2011/0271341	12/2010	Satish et al.	N/A	N/A
2011/0276597	12/2010	Little	709/227	H04L 63/1416
2011/0288940	12/2010	Hordan et al.	N/A	N/A
2012/0023572	12/2011	William, Jr. et al.	N/A	N/A
2012/0030745	12/2011	Bauer	N/A	N/A
2012/0030750	12/2011	Bhargava et al.	N/A	N/A
2012/0079596	12/2011	Thomas et al.	N/A	N/A
2012/0084866	12/2011	Stolfo	N/A	N/A
2012/0096516	12/2011	Sobel et al.	N/A	N/A
2012/0106377	12/2011	Sommers et al.	N/A	N/A
2012/0124363	12/2011	Dietrich et al.	N/A	N/A
2012/0137342	12/2011	Hartrell et al.	N/A	N/A
2012/0137367	12/2011	Dupont et al.	N/A	N/A
2012/0144488	12/2011	Sankruthi	N/A	N/A
2012/0151565	12/2011	Fiterman	N/A	N/A
2012/0185563	12/2011	Sugiyama et al.	N/A	N/A
2012/0240182	12/2011	Marayanaswamy et al.	N/A	N/A
2012/0254995	12/2011	Sallam	N/A	N/A
2012/0255003	12/2011	Sallam	N/A	N/A
2012/0255004	12/2011	Sallam	N/A	N/A
2012/0255012	12/2011	Sallam	N/A	N/A
2012/0255031	12/2011	Sallam	N/A	N/A
2012/0291090	12/2011	Srinivasan et al.	N/A	N/A
2012/0297486	12/2011	Turbin	N/A	N/A
2012/0297488	12/2011	Kapoor et al.	N/A	N/A
2012/0324094	12/2011	Wyatt et al.	N/A	N/A
2012/0331553	12/2011	Aziz et al.	N/A	N/A
2013/0052992	12/2012	Lee	455/411	G06F 21/45
2013/0054682	12/2012	Malik et al.	N/A	N/A
2013/0061097	12/2012	Mendel et al.	N/A	N/A
2013/0080641	12/2012	Lui et al.	N/A	N/A
2013/0086684	12/2012	Mohler	N/A	N/A
2013/0091573	12/2012	Herz et al.	N/A	N/A
2013/0097706	12/2012	Titonis et al.	N/A	N/A
2013/0111547	12/2012	Kraemer	N/A	N/A
2013/0133072	12/2012	Kraitsman	726/23	H04L 63/1433
2013/0152200	12/2012	Alme et al.	N/A	N/A
2013/0167236	12/2012	Sick	N/A	N/A
2013/0191924	12/2012	Tedesco et al.	N/A	N/A
2013/0198842	12/2012	Klein et al.	N/A	N/A
2013/0212658	12/2012	Amaya et al.	N/A	N/A
2013/0219217	12/2012	Seren et al.	N/A	N/A
2013/0231084	12/2012	Raleigh	N/A	N/A
2013/0239192	12/2012	Linga et al.	N/A	N/A
2013/0242743	12/2012	Thomas et al.	N/A	N/A
2013/0246685	12/2012	Bhargava et al.	N/A	N/A
2013/0247190	12/2012	Spurlock	N/A	N/A

2013/0290662	12/2012	Teal	N/A	N/A
2013/0290729	12/2012	Pettigrew	713/189	G06F 21/51
2013/0291111	12/2012	Zhou et al.	N/A	N/A
2013/0298192	12/2012	Kumar et al.	N/A	N/A
2013/0298244	12/2012	Kumar et al.	N/A	N/A
2013/0305377	12/2012	Herz	N/A	N/A
2013/0329732	12/2012	Vyas et al.	N/A	N/A
2013/0333040	12/2012	Diehl et al.	N/A	N/A
2013/0340033	12/2012	Jones et al.	N/A	N/A
2013/0346472	12/2012	Wheeldon	N/A	N/A
2013/0347052	12/2012	Choudrie	726/1	G06F 21/577
2014/0020046	12/2013	Heitzman	N/A	N/A
2014/0046645	12/2013	White et al.	N/A	N/A
2014/0053267	12/2013	Klein et al.	N/A	N/A
2014/0059525	12/2013	Jawa et al.	N/A	N/A
2014/0068326	12/2013	Quinn	N/A	N/A
2014/0068779	12/2013	Tan et al.	N/A	N/A
2014/0090061	12/2013	Avasarala et al.	N/A	N/A
2014/0096229	12/2013	Burns et al.	N/A	N/A
2014/0108794	12/2013	Barton et al.	N/A	N/A
2014/0114609	12/2013	Maurer et al.	N/A	N/A
2014/0123280	12/2013	Kedma	N/A	N/A
2014/0137246	12/2013	Baluda	N/A	N/A
2014/0150094	12/2013	Rao et al.	N/A	N/A
2014/0157366	12/2013	Ko et al.	N/A	N/A
2014/0165203	12/2013	Friedrichs et al.	N/A	N/A
2014/0196108	12/2013	Barr et al.	N/A	N/A
2014/0215617	12/2013	Smith et al.	N/A	N/A
2014/0215621	12/2013	Xaypanya et al.	N/A	N/A
2014/0215625	12/2013	Paul et al.	N/A	N/A
2014/0237562	12/2013	Nandakumar	N/A	N/A
2014/0237595	12/2013	Sridhara et al.	N/A	N/A
2014/0237599	12/2013	Gertner et al.	N/A	N/A
2014/0245376	12/2013	Hibbert et al.	N/A	N/A
2014/0250524	12/2013	Meyers et al.	N/A	N/A
2014/0259092	12/2013	Boucher et al.	N/A	N/A
2014/0270157	12/2013	Youd et al.	N/A	N/A
2014/0280864	12/2013	Yin et al.	N/A	N/A
2014/0282816	12/2013	Xie et al.	N/A	N/A
2014/0283038	12/2013	Call et al.	N/A	N/A
2014/0283076	12/2013	Muttik	N/A	N/A
2014/0289851	12/2013	Klein et al.	N/A	N/A
2014/0298419	12/2013	Boubez et al.	N/A	N/A
2014/0349611	12/2013	Kant et al.	N/A	N/A
2014/0373091	12/2013	Kirner	726/1	H04L 63/1416
2015/0006384	12/2014	Shaikh	N/A	N/A
2015/0007312	12/2014	Pidathala et al.	N/A	N/A
2015/0013006	12/2014	Shulman et al.	N/A	N/A

2015/0013008	12/2014	Lukacs et al.	N/A	N/A
2015/0033341	12/2014	Schmidtler et al.	N/A	N/A
2015/0039513	12/2014	Adjaoute	N/A	N/A
2015/0067763	12/2014	Dalcher et al.	N/A	N/A
2015/0074744	12/2014	McLean et al.	N/A	N/A
2015/0074810	12/2014	Saher et al.	N/A	N/A
2015/0082430	12/2014	Sridhara et al.	N/A	N/A
2015/0089655	12/2014	Choi et al.	N/A	N/A
2015/0096048	12/2014	Zhang et al.	N/A	N/A
2015/0113616	12/2014	Sampas	N/A	N/A
2015/0121524	12/2014	Fawaz et al.	N/A	N/A
2015/0121529	12/2014	Quinlan et al.	N/A	N/A
2015/0128206	12/2014	Haim et al.	N/A	N/A
2015/0128246	12/2014	Feghali et al.	N/A	N/A
2015/0143496	12/2014	Thomas et al.	N/A	N/A
2015/0150125	12/2014	Dulkin et al.	N/A	N/A
2015/0150130	12/2014	Fiala	N/A	N/A
2015/0156214	12/2014	Kaminsky	N/A	N/A
2015/0161024	12/2014	Gupta et al.	N/A	N/A
2015/0163121	12/2014	Mahaffey et al.	N/A	N/A
2015/0172300	12/2014	Cochenour	N/A	N/A
2015/0195291	12/2014	Zuk et al.	N/A	N/A
2015/0199512	12/2014	Kim et al.	N/A	N/A
2015/0200928	12/2014	Burch et al.	N/A	N/A
2015/0200955	12/2014	Martin	N/A	N/A
2015/0205962	12/2014	Swidowski et al.	N/A	N/A
2015/0220735	12/2014	Paithane et al.	N/A	N/A
2015/0254161	12/2014	Baril et al.	N/A	N/A
2015/0257194	12/2014	Cho	N/A	N/A
2015/0264068	12/2014	Beauchesne	N/A	N/A
2015/0264077	12/2014	Berger et al.	N/A	N/A
2015/0268947	12/2014	Ionescu	N/A	N/A
2015/0268989	12/2014	Busch et al.	N/A	N/A
2015/0271200	12/2014	Brady et al.	N/A	N/A
2015/0281267	12/2014	Danahy et al.	N/A	N/A
2015/0286820	12/2014	Sridhara et al.	N/A	N/A
2015/0288706	12/2014	Marshall	N/A	N/A
2015/0310196	12/2014	Turgeman et al.	N/A	N/A
2015/0326587	12/2014	Vissamsetty et al.	N/A	N/A
2015/0326588	12/2014	Vissamsetty et al.	N/A	N/A
2015/0326592	12/2014	Vissamsetty et al.	N/A	N/A
2015/0326599	12/2014	Vissamsetty et al.	N/A	N/A
2015/0332043	12/2014	Russello	N/A	N/A
2015/0346066	12/2014	Dutta et al.	N/A	N/A
2015/0350213	12/2014	Varadarajan	713/189	H04L 63/06
2015/0350236	12/2014	Klinghofer et al.	N/A	N/A
2015/0358345	12/2014	Clark et al.	N/A	N/A
2015/0370560	12/2014	Tan et al.	N/A	N/A
2015/0373039	12/2014	Wang	N/A	N/A
2015/0379371	12/2014	Yoon et al.	N/A	N/A

2015/0381376	12/2014	Wardman et al.	N/A	N/A
2016/0028750	12/2015	Di Pietro et al.	N/A	N/A
2016/0042179	12/2015	Weingarten	N/A	N/A
2016/0042180	12/2015	Sayre et al.	N/A	N/A
2016/0055334	12/2015	Herwono et al.	N/A	N/A
2016/0055337	12/2015	El-Moussa	N/A	N/A
2016/0065675	12/2015	Brand	N/A	N/A
2016/0072838	12/2015	Kolton et al.	N/A	N/A
2016/0078365	12/2015	Baumard	N/A	N/A
2016/0080414	12/2015	Kolton et al.	N/A	N/A
2016/0127352	12/2015	Xu et al.	N/A	N/A
2016/0127413	12/2015	Kraitsman et al.	N/A	N/A
2016/0142399	12/2015	Pace et al.	N/A	N/A
2016/0191554	12/2015	Kaminsky	N/A	N/A
2016/0212225	12/2015	Smith et al.	N/A	N/A
2016/0261631	12/2015	Vissamsetty et al.	N/A	N/A
2016/0285914	12/2015	Singh et al.	N/A	N/A
2016/0315909	12/2015	Von Gravrock et al.	N/A	N/A
2016/0323300	12/2015	Boss et al.	N/A	N/A
2016/0323316	12/2015	Kolton et al.	N/A	N/A
2016/0335110	12/2015	Paithane	N/A	H04L 63/145
2016/0359915	12/2015	Gupta et al.	N/A	N/A
2016/0381023	12/2015	Dulce et al.	N/A	N/A
2017/0009391	12/2016	Fu et al.	N/A	N/A
2017/0019425	12/2016	Ettema et al.	N/A	N/A
2017/0026387	12/2016	Vissamsetty et al.	N/A	N/A
2017/0032122	12/2016	Thakar et al.	N/A	N/A
2017/0054754	12/2016	Saher et al.	N/A	N/A
2017/0093910	12/2016	Gukal et al.	N/A	N/A
2017/0126718	12/2016	Baradaran et al.	N/A	N/A
2017/0134405	12/2016	Ahmadzadeh et al.	N/A	N/A
2017/0141980	12/2016	Palanciuc et al.	N/A	N/A
2017/0147796	12/2016	Sardesai et al.	N/A	N/A
2017/0149787	12/2016	Niemela	N/A	G06F 21/566
2017/0149795	12/2016	Day, II	N/A	N/A
2017/0149832	12/2016	Touboul et al.	N/A	N/A
2017/0171244	12/2016	Vissamsetty et al.	N/A	N/A
2017/0180421	12/2016	Shieh et al.	N/A	N/A
2017/0201545	12/2016	Nicodemus et al.	N/A	N/A
2017/0206142	12/2016	Pawar et al.	N/A	N/A
2017/0206357	12/2016	Gorelik et al.	N/A	N/A
2017/0230384	12/2016	Touboul et al.	N/A	N/A
2017/0230402	12/2016	Greenspan et al.	N/A	N/A
2017/0235967	12/2016	Ray et al.	N/A	N/A
2017/0244729	12/2016	Fahrny et al.	N/A	N/A
2017/0244749	12/2016	Shulman et al.	N/A	N/A
2017/0244755	12/2016	Tsao	N/A	N/A
2017/0250855	12/2016	Patil et al.	N/A	N/A

2017/0257375	12/2016	Dulkin et al.	N/A	N/A
2017/0264639	12/2016	Sama et al.	N/A	N/A
2017/0279839	12/2016	Vasseur et al.	N/A	N/A
2017/0279846	12/2016	Osterweil et al.	N/A	N/A
2017/0286676	12/2016	Weingarten et al.	N/A	N/A
2017/0302458	12/2016	Berger et al.	N/A	N/A
2017/0302653	12/2016	Ortner et al.	N/A	N/A
2017/0302665	12/2016	Zou et al.	N/A	N/A
2017/0302696	12/2016	Schutz et al.	N/A	N/A
2017/0318054	12/2016	Vissamsetty et al.	N/A	N/A
2017/0322959	12/2016	Tidwell et al.	N/A	N/A
2017/0324774	12/2016	Ohayon et al.	N/A	N/A
2017/0324777	12/2016	Ohayon et al.	N/A	N/A
2017/0331849	12/2016	Yu et al.	N/A	N/A
2017/0331856	12/2016	Vissamsetty et al.	N/A	N/A
2017/0346802	12/2016	Gruskin et al.	N/A	N/A
2017/0346853	12/2016	Wyatt et al.	N/A	N/A
2017/0359370	12/2016	Humphries et al.	N/A	N/A
2018/0013788	12/2017	Vissamsetty et al.	N/A	N/A
2018/0020005	12/2017	Beiter et al.	N/A	N/A
2018/0027006	12/2017	Zimmermann et al.	N/A	N/A
2018/0027009	12/2017	Santos et al.	N/A	N/A
2018/0027017	12/2017	Touboul et al.	N/A	N/A
2018/0039776	12/2017	Loman et al.	N/A	N/A
2018/0048665	12/2017	Shulman et al.	N/A	N/A
2018/0063187	12/2017	St Pierre	N/A	N/A
2018/0089430	12/2017	Mayo	N/A	N/A
2018/0089431	12/2017	Nalluri et al.	N/A	N/A
2018/0091553	12/2017	Mandyam et al.	N/A	N/A
2018/0124096	12/2017	Schwartz et al.	N/A	N/A
2018/0143915	12/2017	Gonzalez et al.	N/A	N/A
2018/0146008	12/2017	Vissamsetty et al.	N/A	N/A
2018/0173876	12/2017	Vissamsetty et al.	N/A	N/A
2018/0183815	12/2017	Enfinger	N/A	N/A
2018/0191593	12/2017	De et al.	N/A	N/A
2018/0212999	12/2017	Kassimis et al.	N/A	N/A
2018/0225592	12/2017	Ponnuswamy	N/A	N/A
2018/0248896	12/2017	Challita et al.	N/A	N/A
2018/0343281	12/2017	Ahuja et al.	N/A	N/A
2018/0359272	12/2017	Mizrachi et al.	N/A	N/A
2018/0359278	12/2017	Rusakov et al.	N/A	N/A
2019/0042745	12/2018	Chen et al.	N/A	N/A
2019/0052659	12/2018	Weingarten et al.	N/A	N/A
2019/0068642	12/2018	Araujo et al.	N/A	N/A
2019/0073475	12/2018	Vissamsetty et al.	N/A	N/A
2019/0089677	12/2018	Ashley et al.	N/A	N/A
2019/0114426	12/2018	Cohen et al.	N/A	N/A
2019/0138639	12/2018	Pal et al.	N/A	N/A
2019/0138725	12/2018	Gupta	N/A	N/A
2019/0199736	12/2018	Howard et al.	N/A	N/A

2019/0253453	12/2018	Vissamsetty et al.	N/A	N/A
2019/0258807	12/2018	DiMaggio et al.	N/A	N/A
2019/0294485	12/2018	Kukreja et al.	N/A	N/A
2019/0294614	12/2018	Brodt et al.	N/A	N/A
2019/0312904	12/2018	Stevens, IV et al.	N/A	N/A
2019/0318100	12/2018	Bhatia et al.	N/A	N/A
2019/0340353	12/2018	Mitelman	N/A	N/A
2019/0349369	12/2018	Bengtson et al.	N/A	N/A
2019/0354355	12/2018	Jacobson et al.	N/A	N/A
2019/0379697	12/2018	Vissamsetty et al.	N/A	N/A
2019/0392260	12/2018	Pang et al.	N/A	N/A
2020/0050586	12/2019	Pal et al.	N/A	N/A
2020/0051697	12/2019	Krishnamurti et al.	N/A	N/A
2020/0076853	12/2019	Pandian et al.	N/A	N/A
2020/0133865	12/2019	Mannava et al.	N/A	N/A
2020/0137114	12/2019	Bender et al.	N/A	N/A
2020/0143054	12/2019	Cohen et al.	N/A	N/A
2020/0143061	12/2019	Kim et al.	N/A	N/A
2020/0201620	12/2019	Beard	N/A	N/A
2020/0218806	12/2019	Cho	N/A	N/A
2020/0244677	12/2019	Abbaszadeh et al.	N/A	N/A
2020/0252429	12/2019	Vissamsetty et al.	N/A	N/A
2020/0274894	12/2019	Argoeti et al.	N/A	N/A
2020/0351245	12/2019	Moore et al.	N/A	N/A
2020/0366686	12/2019	Gal et al.	N/A	N/A
2020/0374087	12/2019	Vissamsetty et al.	N/A	N/A
2020/0389432	12/2019	Panchalingam et al.	N/A	N/A
2021/0021572	12/2020	Bonczar	N/A	N/A
2021/0029153	12/2020	Sugimoto et al.	N/A	N/A
2021/0037046	12/2020	Dhillon	N/A	N/A
2021/0044623	12/2020	Bosch et al.	N/A	N/A
2021/0073374	12/2020	Mookken et al.	N/A	N/A
2021/0073658	12/2020	Poole et al.	N/A	N/A
2021/0117232	12/2020	Sriharsha et al.	N/A	N/A
2021/0185538	12/2020	Zhang et al.	N/A	N/A
2021/0203699	12/2020	Schmugar	N/A	N/A
2021/0232291	12/2020	Abdulaal et al.	N/A	N/A
2021/0263945	12/2020	Siebel et al.	N/A	N/A
2021/0397696	12/2020	Yarabolu	N/A	N/A
2021/0397710	12/2020	Cohen et al.	N/A	N/A
2022/0050895	12/2021	Yu et al.	N/A	N/A
2022/0070256	12/2021	Singh et al.	N/A	N/A
2022/0086142	12/2021	Hecht	N/A	N/A
2022/0101096	12/2021	Singer et al.	N/A	N/A
2022/0164444	12/2021	Prudkovskij	N/A	N/A
2022/0253526	12/2021	Sanders	N/A	N/A
2022/0391496	12/2021	Salem et al.	N/A	N/A
2022/0414676	12/2021	Power et al.	N/A	N/A
2023/0007025	12/2022	Weingarten et al.	N/A	N/A
2023/0007026	12/2022	Weingarten et al.	N/A	N/A

2023/0007027	12/2022	Weingarten et al.	N/A	N/A
2023/0007028	12/2022	Weingarten et al.	N/A	N/A
2023/0007029	12/2022	Weingarten et al.	N/A	N/A
2023/0007030	12/2022	Weingarten et al.	N/A	N/A
2023/0007031	12/2022	Weingarten et al.	N/A	N/A
2023/0053937	12/2022	Choi et al.	N/A	N/A
2023/0247046	12/2022	Peters et al.	N/A	N/A
2023/0247048	12/2022	Samosseiko et al.	N/A	N/A
2023/0291769	12/2022	Talur et al.	N/A	N/A
2023/0319089	12/2022	Chesla et al.	N/A	N/A
2023/0336573	12/2022	Jones	N/A	N/A
2023/0362187	12/2022	Walch et al.	N/A	N/A

FOREIGN PATENT DOCUMENTS

Patent No.	Application Date	Country	CPC
103607399	12/2013	CN	N/A
109446755	12/2018	CN	N/A
113297584	12/2020	CN	N/A
113434858	12/2020	CN	N/A
3 171 568	12/2016	EP	N/A
3472746	12/2018	EP	N/A
3968197	12/2021	EP	N/A
2785350	12/2019	ES	N/A
2006-106939	12/2005	JP	N/A
2008-252625	12/2007	JP	N/A
2013-168141	12/2012	JP	N/A
2015-534690	12/2014	JP	N/A
2016-512631	12/2015	JP	N/A
2017-504102	12/2016	JP	N/A
10-2015-0101811	12/2014	KR	N/A
10-1969572	12/2018	KR	N/A
10-2021-0079494	12/2020	KR	N/A
WO 02/27440	12/2001	WO	N/A
WO 2010/030169	12/2009	WO	N/A
WO 2012/027669	12/2011	WO	N/A
WO 2013/014672	12/2012	WO	N/A
2013/121951	12/2012	WO	N/A
WO 2015/171780	12/2014	WO	N/A
WO 2015/171789	12/2014	WO	N/A
WO 2016/024268	12/2015	WO	N/A
WO 2016/081561	12/2015	WO	N/A
2017/068889	12/2016	WO	N/A
WO 2017/064710	12/2016	WO	N/A
2017/218872	12/2016	WO	N/A
WO 2019/092530	12/2018	WO	N/A
WO 2019/032728	12/2019	WO	N/A
2020/108357	12/2019	WO	N/A
2022/076234	12/2021	WO	N/A

OTHER PUBLICATIONS

Chen et al., “MitM attack by name collision: Cause analysis and vulnerability assessment in the new gTLD era.” Security and Privacy (SP), 2016 IEEE Symposium on. IEEE (2016). cited by applicant

Dini et al., “Probabilistic Contract Compliance for Mobile Applications”, Eighth International Conference on Availability, Reliability and Security (ARES) IEEE, Sep. 2-6, 2013, pp. 599-606. cited by applicant

Gu et al., “Malicious Shellcode Detection with Virtual Memory Snapshots,” 2010 Proceedings IEEE Infocom, San Diego, CA, 2010, pp. 1-9, accessed Mar. 6, 2019. cited by applicant

Installing a Tanium Client—distributed also using user/password (not encrypted) from a hosted Module server, downloaded from https://docs.tanium.com/client/client/deployment.html#client_management on Apr. 1, 2021. cited by applicant

IBM Software, “Stepping up the battle against advanced threats”, IBM Trusteer Apex Thought Leadership White Paper, Dec. 2013, WGW03043-USEN-00. cited by applicant

“IBM Security Trusteer Apex Advanced malware Protection (SaaS) delivers an enterprise endpoint security solution to help protect organizations from advanced malware and targeted attacks” downloaded from https://www-01.ibm.com/common/ssi/rep_ca/0/877/ENUSZP14-0390/ENUSZP14-0390.PDF, printed Aug. 5, 2014, in 9 pgs. cited by applicant

IBM Guardium Documentation “Components and Topology”, downloaded from <https://www.ibm.com/docs/en/qsip/7.4?topic=deployment-qradar-architecture-overview>, printed on Jun. 8, 2021 in 3 pages. cited by applicant

IBM Software “Stepping up the battle against advanced threats”, downloaded from <https://citrixready.citrix.com/content/dam/ready/partners/ib/ibm-global-services/IBM-security-trusteer-apex-advanced-malware-protection/wgw03043usen.pdf>, Printed Dec. 2013 in 12 pages. cited by applicant

Laureano et al., M., “Intrusion detection in virtual machine environments. In Euromicro Conference, 2004. Proceedings.” 30th (pp. 520-525). IEEE Sep. 30, 2004. cited by applicant

Liu et al., “Detecting Trojan Horses Based on System Behavior Using Machine Learning Method”, International Conference on Machine Learning and Cybernetics (ICMLC), IEEE, Jul. 11-14, 2010, pp. 855-860. cited by applicant

Ibm, “Qradar Architecture overview”, downloaded from <https://www.ibm.com/docs/en/qsip/7.4?topic=deployment-qradar-architecture-overview> printed May 28, 2021 in 6 pgs. cited by applicant

Ornaghi et al., “Man in the middle attacks.” Blackhat Conference Europe (2003). cited by applicant

Ramachandran et al., “Detecting ARP spoofing: An active technique.” International Conference on Information Systems Security, Springer, Berlin, Heidelberg (2005). cited by applicant

Rüdiger Schollmeier, A Definition of Peer-to-Peer Networking for the Classification of Peer-to-Peer Architectures and Applications, Proceedings of the First International Conference on Peer-to-Peer Computing, IEEE (2002). cited by applicant

Shosha et al., A.F., “Evasion-resistant malware signature based on profiling kernel data structure objects.” In 2012 7th International Conference on Risks and Security of Internet and Systems (CRISIS) (pp. 1-8). IEEE., Oct. 31, 2012. cited by applicant

Tanium™ Client Management 1.6 User Guide, downloaded from https://docs.tanium.com/client_management/client_management/index.html on Apr. 1, 2021. cited by applicant

Xu, J-Y; Sung, A.H.; Chavez, P.; Mukkamala, S.; “Polymorphic Malicious Executable Scanner by API Sequence Analysis”, Fourth International Conference on Hybrid Intelligent Systems, IEEE Dec. 5-8, 2004, pp. 378-383. cited by applicant

Communication Pursuant to Article 94(3) EPC dated Dec. 11, 2018 for European Application 15 760 520.5, in 7 pages. cited by applicant

Extended European Search Report dated May 9, 2019 for Application No. 16855062.2, in 7 pages. cited by applicant

European Search Report dated Apr. 29, 2021 in European Patent Application No. 18844671 in 38 pages. cited by applicant

Extended European Search Report dated Jan. 25, 2021 for European Patent Application No. 20181537.0, in 10 pages. cited by applicant

Reaqta Hive, A.I. Based Endpoint Threat Response, Whitepaper, 27 pages (Apr. 6, 2017). cited by applicant

International Search Report and Written Opinion dated Feb. 18, 2016 for International Application No. PCT/IL2015/050802, in 10 pages. cited by applicant

International Preliminary Report on Patentability dated Feb. 14, 2017 for International Application No. PCT/IL2015/050802, in 7 pages. cited by applicant

International Search Report and Written Opinion dated Apr. 20, 2017 for International Application No. PCT/IL2016/051110, in 10 pages. cited by applicant

International Search Report and Written Opinion dated Dec. 11, 2018 for International Application No. PCT/US2018/045850, in 12 pages. cited by applicant

International Search Report and Written Opinion dated Aug. 24, 2020 for International Application No. PCT/US2020/033872, in 8 pages. cited by applicant

International Search Report issued in application No. PCT/US2021/050129 on Dec. 21, 2021. cited by applicant

International Search Report in corresponding International Patent Application No. PCT/US2015/29490, dated Aug. 7, 2015, in 2 pages. cited by applicant

International Preliminary Report on Patentability in corresponding International Patent Application No. PCT/US2015/29490, dated Nov. 8, 2016, in 7 pages. cited by applicant

International Search Report in corresponding International Patent Application No. PCT/US2015/29501, dated Aug. 7, 2015, in 2 pages. cited by applicant

International Preliminary Report on Patentability in corresponding International Patent Application No. PCT/US2015/29501, dated Nov. 8, 2016, in 7 pages. cited by applicant

Written Opinion in corresponding International Patent Application No. PCT/US2015/61271, dated Feb. 2, 2016, in 6 pages. cited by applicant

International Preliminary Report on Patentability in corresponding International Patent Application No. PCT/US2015/61271, dated May 23, 2017, in 7 pages. cited by applicant

Office Action dated Jul. 24, 2019 in European Patent Application No. 15760520.5, in 8 pages. cited by applicant

Office Action dated May 31, 2022 in Japanese Patent Application No. 2020-503272, in 7 pages. cited by applicant

Extended European Search Report dated Aug. 25, 2021 for European Patent Application No. 21162973.8, in 5 pages. cited by applicant

Office Action dated Aug. 1, 2023 in U.S. Appl. No. 18/183,022, 16 pages. cited by applicant

Office Action dated Sep. 8, 2023 in U.S. Appl. No. 18/173,611, 10 pages. cited by applicant

Notice of Allowance dated Jan. 30, 2024 in U.S. Appl. No. 18/183,022, 10 pages. cited by applicant

AEfficient Virus Detection Using Dynamic Instruction Sequences—May 2009 (Year: 2009). cited by applicant

“IBM Security Qradar SIEM Installation Guide” downloaded from [http://www.](http://www.siem.su/docs/IBM/Installation_and Updating/IBM_Security_QRadar_installation_Guide.pdf)

[siem.su/docs/IBM/Installation_and Updating/IBM_Security_QRadar_installation_Guide.pdf](http://www.siem.su/docs/IBM/Installation_and Updating/IBM_Security_QRadar_installation_Guide.pdf), printed 2013, vol. 7.2 in 54 pages. cited by applicant

Abdelhameed, M.U, et al. Portable executable automatic protection using dynamic infection and code redirection; 2009 International Conference on Computer Engineering & Systems (pp. 501-507); (Year: 2009). cited by applicant

Abrath, Bert et al. Obfuscating Windows DLLs; 2015 IEEE/ACM 1st International Workshop on

Software Protection (pp. 24-30); (Year: 2015). cited by applicant

Barbhuiya et al., "Detection of neighbor solicitation and advertisement spoofing in 1Pv6 neighbor discovery protocol." Proceedings of the 4th international conference on Security of information and networks. (2011). cited by applicant

Birrer, B, et al., "Using qualia and hierarchical models in malware detection", Journal of Information Assurance and Security, Dec. 29, 2009, pp. 247-255. cited by applicant

Chakraborty et al., "Hierarchical learning for automated malware classification", MILCOM 2017-2017 IEEE Military Communications Conference (MILCOM), Oct. 23-25, 2017, pp. 1-6. cited by applicant

IBM Security Qradar Solutuion Brief "Visibility, detection, investigation and response" printed from <https://www.ibm.com/downloads/cas/OP62GKAR>, printed on May 26, 2021 in 11 pgs. cited by applicant

International Search Report in corresponding International Patent Application No. PCT/US2015/61271, dated Feb. 2, 2016, in 2 pages. cited by applicant

Jack Ullrich, Detecting Manual Syscalls from User Mode, Winternl, Cybersecurity Research & Development, Feb. 10, 2021, 11 pages, <https://winternl.com/detecting-manual-syscalls-from-user-mode/>. cited by applicant

Mahler, "A New Methodology for Information Security Risk Assessment for Medical Devices and Its Evaluation," 2020, The Department of Software and Information Systems Engineering, Ben-Gurion University of the Negev, Israel. cited by applicant

Matsuda, Wataru et al. "Real-Time Detection System Against Malicious Tools by Monitoring DLL on Client Computers;" 2019 IEEE Conference on Application, Information and Network Security (AINS) (pp. 36-41); (Year: 2019). cited by applicant

Moussaileb et al., "Ransomware's Early Mitigation Mechanisms," Proceedings ARES 2018 Proceedings of the 13th International Conference on Availability, Reliability and Security Article No. 2, pp. 1-10 (2018). cited by applicant

Nisslmueller, U., "LOLBin detection through unsupervised learning An approach based on explicit featurization of the command line and parent-child relationships", University of Twente, Student Theses, Sep. 27, 2022, 72 pages. cited by applicant

Pandian, "Security Challenges of IoT and Medical Devices in Healthcare," Internet of Things, 1st Edition, 2020, Chapter 6, pp. 87-106. cited by applicant

Shun-Wen Hsiao et al., "Sequence Feature Extraction for Malware Family Analysis via Graph Neural," Network, arXiv:2208.05476v1, Aug. 10, 2022, pp. 1-12. cited by applicant

Stine, "A cyber risk scoring system for medical devices," International Journal of Critical Infrastructure Protection, Dec. 2017, vol. 19, pp. 32-46. cited by applicant

Written Opinion in corresponding International Patent Application No. PCT/US2015/29490, dated Aug. 7, 2015, in 6 pages. cited by applicant

Written Opinion in corresponding International Patent Application No. PCT/US2015/29501, dated Aug. 7, 2015, in 6 pages. cited by applicant

Zaldivar, "Investigating the Security Threats on Networked Medical Devices," 2020, 10th Annual Computing and Communication Workshop and Conference, Jan. 2020. cited by applicant

IBM Security Guardium Insights for IBM Cloud Park for Security, printed from <https://www.ibm.com/downloads/cas/QY1RYRLP>, printed May 26, 2021 in 12 pages. cited by applicant

Barbhuiya et al., "Detection of neighbor solicitation and advertisement spoofing in IPV6 neighbor discovery protocol." Proceedings of the 4th international conference on Security of information and networks. (2011). cited by applicant

Installing a Tanium Client—distributed also using user/password (not encrypted) from a hosted Module server, downloaded from https://docs.tanium.com/client/client/deployment.html#client_management on Apr. 2021. cited by

applicant

Moussaileb et al., “Ransomware's Early Mitigation Mechanisms,” Proceedings ARES 2018 Proceedings of the 13th International Conference on Availability, Reliability and Security Article No. 2, 02:S 1-10 (2018). cited by applicant

Ullrich et al., “IPv6 Security: Attacks and Countermeasures in a Nutshell.” 8th USENIX Workshop on Offensive Technologies (2015). cited by applicant

Bert Abrath · Bart Coppens · Stijn Volckaert · Bjorn De Sutter; Obfuscating Windows DLLs; 2015 IEEE/ACM 1st International Workshop on Software Protection (2015, pp. 24-30); (Year: 2015). cited by applicant

International Search Report and Written Opinion received for PCT Patent Application No. PCT/US24/44217, mailed on Jan. 15, 2025, 24 pages. cited by applicant

U.S. Appl. No. 61/989,965, filed May 7, 2014, “Distributed System for Bot Detection”, 54 pages. cited by applicant

Primary Examiner: Huang; Cheng-Feng

Attorney, Agent or Firm: Perkins Coie LLP

Background/Summary

RELATED APPLICATIONS (1) Any and all applications for which a foreign or domestic priority claim is identified in the Application Data Sheet as filed with the present application are hereby incorporated by reference under 37 CFR 1.57. (2) This application is a continuation of U.S. application Ser. No. 16/849,813, filed Apr. 15, 2020, and entitled “Deceiving Attackers Accessing Network Data” which is a continuation-in-part of U.S. application Ser. No. 16/543,189, filed Aug. 16, 2019, and entitled “Deceiving Attackers Accessing Active Directory Data”, which is hereby incorporated herein by reference in its entirety. U.S. patent application Ser. No. 16/543,189 is a continuation-in-part of U.S. application Ser. No. 15/383,522, filed Dec. 19, 2016, and entitled “Deceiving Attackers in Endpoint Systems”, which is hereby incorporated herein by reference in its entirety for all purposes.

BACKGROUND

(1) Once an end point system is compromised, attackers try to move laterally in the network. Attackers harvest data from end point systems and then use that information to move laterally. The systems and methods disclosed herein provide an improved approach for preventing unauthorized access to application data on endpoint systems.

Description

BRIEF DESCRIPTION OF THE FIGURES

(1) In order that the advantages of the invention will be readily understood, a more particular description of the invention briefly described above will be rendered by reference to specific embodiments illustrated in the appended drawings. Understanding that these drawings depict only typical embodiments of the invention and are not therefore to be considered limiting of its scope, the invention will be described and explained with additional specificity and detail through use of the accompanying drawings, in which:

(2) FIG. 1 is a schematic block diagram of a network environment for performing methods in accordance with an embodiment of the present invention;

- (3) FIG. 2 is a diagram illustrating components for preventing unauthorized access to application data in accordance with an embodiment of the present invention;
- (4) FIGS. 3A and 3B are process flow diagrams of methods for preventing unauthorized access in accordance with an embodiment of the present invention;
- (5) FIG. 4 is a schematic block diagram of an alternative network environment for performing methods in accordance with an embodiment of the present invention;
- (6) FIG. 5 is a diagram illustrating components for preventing unauthorized access to active directory data in accordance with an embodiment of the present invention;
- (7) FIG. 6 is a process flow diagram of a method for preventing unauthorized access to active directory data in accordance with an embodiment of the present invention;
- (8) FIG. 7 is a diagram illustrating components for preventing unauthorized access to domain data in accordance with an embodiment of the present invention;
- (9) FIG. 8 is a process flow diagram of a method for preventing unauthorized access to domain data in accordance with an embodiment of the present invention;
- (10) FIG. 9A is a schematic block diagram illustrating exploitation of an active directory server using an infected endpoint;
- (11) FIG. 9B is a schematic block diagram illustrating prevention of exploitation of an active directory server using an infected endpoint in accordance with an embodiment of the present invention;
- (12) FIG. 10 is a process flow diagram illustrating the use of decoy session data in accordance with an embodiment of the present invention;
- (13) FIG. 11 is a schematic block diagram of configuration of dynamic link libraries in accordance with an embodiment of the present invention;
- (14) FIG. 12 is a process flow diagram of a method for intercepting system calls in accordance with an embodiment of the present invention;
- (15) FIG. 13 is a process flow diagram of a method for intercepting calls to list network shares in accordance with an embodiment of the present invention;
- (16) FIG. 14 is a process flow diagram of a method for intercepting calls to list remote network resources in accordance with an embodiment of the present invention;
- (17) FIG. 15 is a process flow diagram of a method for intercepting calls to a database for a privileged group in accordance with an embodiment of the present invention;
- (18) FIG. 16 is a process flow diagram of a method for intercepting access requests with DNS names in accordance with an embodiment of the present invention; and
- (19) FIG. 17 is a schematic block diagram of a computer system suitable for implementing methods in accordance with embodiments of the present invention.

DETAILED DESCRIPTION

(20) It will be readily understood that the components of the invention, as generally described and illustrated in the Figures herein, could be arranged and designed in a wide variety of different configurations. Thus, the following more detailed description of the embodiments of the invention, as represented in the Figures, is not intended to limit the scope of the invention, as claimed, but is merely representative of certain examples of presently contemplated embodiments in accordance with the invention. The presently described embodiments will be best understood by reference to the drawings, wherein like parts are designated by like numerals throughout.

(21) Embodiments in accordance with the invention may be embodied as an apparatus, method, or computer program product. Accordingly, the invention may take the form of an entirely hardware embodiment, an entirely software embodiment (including firmware, resident software, micro-code, etc.), or an embodiment combining software and hardware aspects that may all generally be referred to herein as a “module” or “system.” Furthermore, the invention may take the form of a computer program product embodied in any tangible medium of expression having computer-usable program code embodied in the medium.

(22) Any combination of one or more computer-usable or computer-readable media may be utilized. For example, a computer-readable medium may include one or more of a portable computer diskette, a hard disk, a random access memory (RAM) device, a read-only memory (ROM) device, an erasable programmable read-only memory (EPROM or Flash memory) device, a portable compact disc read-only memory (CDROM), an optical storage device, and a magnetic storage device. In selected embodiments, a computer-readable medium may comprise any non-transitory medium that can contain, store, communicate, propagate, or transport the program for use by or in connection with the instruction execution system, apparatus, or device.

(23) Computer program code for carrying out operations of the invention may be written in any combination of one or more programming languages, including an object-oriented programming language such as Java, Smalltalk, C++, or the like and conventional procedural programming languages, such as the "C" programming language or similar programming languages, and may also use descriptive or markup languages such as HTML, XML, JSON, and the like. The program code may execute entirely on a computer system as a stand-alone software package, on a stand-alone hardware unit, partly on a remote computer spaced some distance from the computer, or entirely on a remote computer or server. In the latter scenario, the remote computer may be connected to the computer through any type of network, including a local area network (LAN) or a wide area network (WAN), or the connection may be made to an external computer (for example, through the Internet using an Internet Service Provider).

(24) The invention is described below with reference to flowchart illustrations and/or block diagrams of methods, apparatus (systems) and computer program products according to embodiments of the invention. It will be understood that each block of the flowchart illustrations and/or block diagrams, and combinations of blocks in the flowchart illustrations and/or block diagrams, can be implemented by computer program instructions or code. These computer program instructions may be provided to a processor of a general purpose computer, special purpose computer, or other programmable data processing apparatus to produce a machine, such that the instructions, which execute via the processor of the computer or other programmable data processing apparatus, create means for implementing the functions/acts specified in the flowchart and/or block diagram block or blocks.

(25) These computer program instructions may also be stored in a non-transitory computer-readable medium that can direct a computer or other programmable data processing apparatus to function in a particular manner, such that the instructions stored in the computer-readable medium produce an article of manufacture including instruction means which implement the function/act specified in the flowchart and/or block diagram block or blocks.

(26) The computer program instructions may also be loaded onto a computer or other programmable data processing apparatus to cause a series of operational steps to be performed on the computer or other programmable apparatus to produce a computer implemented process such that the instructions which execute on the computer or other programmable apparatus provide processes for implementing the functions/acts specified in the flowchart and/or block diagram block or blocks.

(27) Referring to FIG. 1, the methods disclosed herein may be practiced in a network environment **100** including a plurality of domains **102a-102c**. The domains **102a-102c** may be any network division, such as a subnet, local area network (LAN), virtual local area network (VLAN), or the like. The domains **102a-102c** may be distributed within a same building or over a large geographic area with interconnecting links including the Internet **104**. The illustrated domains **102a-102c** may represent one or more network components, such as routers, switches, servers, and the like that implement routing of traffic within the domains **102a-102c** and control traffic flowing into and out of the domains **102a-102c**.

(28) Each domain may include one or more endpoints **106a-106g**. The endpoints **106a-106g** are production computing devices that operate as personal computers for users or servers providing

production services to other endpoints or to external computers accessing the network environment by way of the internet **104**. The endpoints **106a-106g** may be desktop or laptop computers, mobile phones, tablet computers, server computers, and any other type of computing device. Some endpoints **106a-106g** may include internet-enabled devices, i.e. so-called internet of things (IoT) devices that are often a vulnerability.

(29) The endpoints **106a-106g** are not dedicated honeypots, but rather perform non-decoy functions and process legitimate production data and legitimate production tasks of an enterprise, such as functioning as user computers executing applications such as word processors, browsers, graphics programs etc. The endpoints **106a-106g** may also function as web servers, database servers, remote login servers, application servers, and the like.

(30) Some or all of the endpoints **106a-106g** execute a sensor module **108**. The sensor module **108** stores or accesses a list of sanctioned applications **110a** and may also store or access a listing or description of protected data **110b**. The endpoints **106a-106g** may execute one or more instances of one or more of the sanctioned applications **110a** and store data generated or used by these applications that corresponds to the protected data. The sanctioned applications **110a** are production applications for performing any of the production tasks and functionality mentioned above. Accordingly, the protected data **110b** may also be production data for use in performing the production tasks and functionality mentioned above, as opposed to decoy or deceptive data.

(31) The sensor module **108** may store or access deception data **110c** stored locally on the endpoint **106a-106g** or accessed from another location. The deception data **110c** may mimic the format of production data in the form of web pages, word processor documents, spreadsheets, databases, etc. The deception data **110c** may also mimic other files used by applications such as credentials for authenticating the application with a remote server, configuration files, browser histories, a listing of recently accessed files, configuration files, and the like.

(32) In the case of credentials or other files that are used to access a remote server or provide a record of accessing a remote server, the deception data **110c** may reference a BotSink **120**. The BotSink **120** may function as a honey pot programmed to engage an attacker while preventing access to production data or computer systems. For example, the BotSink **120** may execute one or more virtual machines implementing network services that engage and monitor malicious code while preventing access to other endpoints **106a-106g** of the network. The BotSink **120** may implement any of the method methods for detecting and engaging malicious code disclosed in the following applications (herein after “the incorporated applications”), which are hereby incorporated herein by reference in their entirety: U.S. application Ser. No. 14/458,026, filed Aug. 12, 2014, and entitled DISTRIBUTED SYSTEM FOR BOT DETECTION; U.S. application Ser. No. 14/466,646, filed Aug. 22, 2014, and entitled EVALUATING URLS FOR MALICIOUS CONTENT; U.S. application Ser. No. 14/549,112, filed Nov. 20, 2014, and entitled METHOD FOR DIRECTING MALICIOUS ACTIVITY TO A MONITORING SYSTEM; U.S. application Ser. No. 15/157,082, filed May 17, 2016, and entitled EMULATING SUCCESSFUL SHELLCODE ATTACKS; U.S. application Ser. No. 14/805,202, filed Jul. 21, 2015, and entitled MONITORING ACCESS OF NETWORK DARKSPACE; U.S. application Ser. No. 14/965,574, filed Dec. 10, 2015, and entitled DATABASE DECEPTION IN DIRECTORY SERVICES; U.S. application Ser. No. 15/142,860, filed Apr. 29, 2016, and entitled AUTHENTICATION INCIDENT DETECTION AND MANAGEMENT; U.S. application Ser. No. 15/153,471, filed May 12, 2016, and entitled LURING ATTACKERS TOWARDS DECEPTION SERVERS; U.S. application Ser. No. 15/204,779, filed Jul. 7, 2016, and entitled DETECTING MAN-IN-THE-MIDDLE ATTACKS; and U.S. application Ser. No. 15/360,117, filed Nov. 23, 2016, and entitled IMPLEMENTING DECOYS IN NETWORK ENDPOINTS.

(33) In some embodiments, the data **110a-110c** is provided to the endpoints **106a-106g** by a management server **112**. The management server **112** may implement an application policy module **114**. The application policy module **114** stores or accesses a listing **116a** of sanctioned applications

and may provide an interface for an administrator to specify what applications are included in the listing **116a**. The listing **116a** may indicate which applications are sanctioned for particular endpoints **106a-106g** or for endpoints in a particular domain **102a-102c**. The listing **116a** may be automatically updated to include applications as they are installed on endpoints **106a-106g** by an administrator.

(34) In a similar manner, the management server **112** may store or access protected data **116b** that lists data files, folders, or other descriptors of data that are protected. The protected data **116b** may be obtained automatically from configuration files for applications. For example, an application on installation will often create directories for files used by the application. Accordingly, these directories and files may be observed by analyzing the configuration files for instructions to create these directories and files or by observing how the file system changes following installation of the application on a sample endpoint.

(35) The management server **112** may also store or access deception data **118**. As noted above, this deception data may mimic production data for one or more applications and may reference the BotSink **120** in order to lure attackers into engagement with the BotSink **120**.

(36) The management server **112** distributes some or all of the data **116a-116b**, **118** to the endpoints **106a-106g**. For example, deception data **118** provided to an endpoint may mimic files generated or used by one or more production applications actually installed on that endpoint. Likewise, the listing of sanctioned applications **116a** provide to the endpoint may include only those applications that were installed by an administrator or authorized user on that endpoint. The deception data **118** provided to the endpoint may then include deception data mimicking the production data of those applications. The management server **112** may periodically update the data **116a-116b**, **118** and distribute updated data to the endpoints **106a-106g**.

(37) The methods disclosed herein are invoked in response to an attacker system **122** attempting to access production application data on an endpoint **106a-106g**. This may include the attacker system **122** issuing commands to the endpoint **106a-106g** or uploading malicious code to the endpoint **106a-106g**, which then attempts to access the production application data. However, unauthorized access may be prevented using the methods disclosed herein in either case.

(38) Referring to FIG. 2, the sensor module **108** may be incorporated into the operating system **200**, such as by modifying the functions of one or more libraries for performing access to a file system. The sensor module **108** intercepts file system command from instances of applications **202** installed on the endpoint as well as file system commands from an attacker, such as an attacker tool **204** executing on the endpoint. The file system commands that are intercepted may include write commands, read commands, delete commands, or instructions to list the contents of a directory or other commands to navigate through a directory.

(39) The sensor module **108** evaluates a source of each command, e.g. the binary instance executing on the endpoint that issued the command to the operating system. If the source of the command is an application listed in the sanctioned applications **110a**, the command is passed to the file system I/O (input output) functions **206** of the operating system **200**, which then executes the command with respect to the production application data **208**.

(40) If the source of the command is not found to be in the sanctioned applications **110a**, the command may be modified such that it refers to deception data **110c**. The modified command may then be input to the file system I/O functions **206**. In the case of a write or delete command, the sensor module **108** may suppress execution of the command and return an acknowledgment to the source of the command indicating that the command was executed successfully.

(41) In the embodiment of FIG. 2, the modified commands are executed by the same file system I/O functions **206** as other commands. Accordingly, the deception data **110c** may be stored in the file system and accessible to such functions **206**. However, the deception data **110c** may be hidden such that it is not viewable by users or applications executing on the endpoint.

(42) In other embodiments, the sensor module **108** itself may access and return the deception data

110c, in which case the deception data **110c** may be stored anywhere, including remotely from the endpoint and referenced by the sensor module **108**. The deception data **110c** could, for example, be encrypted such that only the sensor module **108** can decrypt the deception data **110c** in order to return it in response to a read command from the attacker tool **204** or other unauthorized source.

(43) In still other embodiments, the sensor module **108** may automatically generate deception data **110c** in response to file system commands, such as based on templates that are populated with random data in order to mimic a type of file requested by the file system command from the attacker tool **204**.

(44) Referring to FIG. 3A, the sensor module **108** may include any executable code programmed to execute the illustrated method **300a**. The method **300a** may include receiving **302** a file system command, such as by intercepting a command made to the operating system **200** of the endpoint in which the sensor module **108** is embedded.

(45) The method **300a** may include evaluating **304** whether the data (file, directory, type of file, etc.) is protected data, such as might be indicated by the listing **110b** of protected data on the endpoint. If not, the method **300a** may include executing **306** the file system command without modification, i.e. passing it to the operating system **200** for execution. In some embodiments, only the sanctioned applications **110a** are permitted to issue file system commands, which may include operating system utilities. Accordingly, in some embodiments, step **304** may be omitted.

(46) The method **300a** may further include evaluating a source of the file system command according to some or all of steps **308-312**. For example, if the source of the file system command is found **308** to have a certificate matching that of a sanctioned application **110a**, then file system command may be executed **306** with respect to the data referenced in the command.

(47) If a hash, e.g. the SHA-1 (secure hash algorithm) hash of the binary code that issued the file system command is found **310** to match a hash of the binary executable for one of the sanctioned applications **110a**, then the file system command may be executed **306** with respect to the data referenced in the command.

(48) If a path to the binary code that issued the file system command is found **312** to match the path to the binary executable of one of the sanctioned applications **110a**, then the file system command may be executed **306** with respect to the data referenced in the command.

(49) Steps **308-312** are just examples of checks that may be used to verify whether binary code issuing a command is in fact an instance of a sanctioned application. In some embodiments, all of steps **308-312** must be satisfied before step **306** will be executed with respect to the data referenced in the command. In other embodiments, other checks may be used as alternatives or as additional requirements before step **306** will be executed with respect to the data referenced in the command.

(50) In some embodiments, certain protected data may be bound to a particular sanctioned application **110a**. Accordingly steps **308-312** may be evaluated only for those applications that are bound to the protected data **110b** referenced in the file system command, i.e. the command will be executed with respect to the data referenced in the command only if one of **308-312** (or each and every one of **308-312** in some embodiments) is satisfied for at least one sanctioned application **110a** that is also bound to the protected data **110b** referenced in the file system command.

(51) If the tests of steps **308-312** are not sufficient to invoke execution of step **306** for the data referenced in the file system command according to any of the embodiments mentioned above, then the source of the file system command may be determined not to be one of the sanctioned applications **110a** and processing continues at steps **314, 318**.

(52) If the file system command is found **314** to be a read command, then file system command may be changed **318** to refer to the deception data **110c** of the endpoint. Step **306** may be executed with respect to the modified file system command, which will include returning the deception data referenced by the modified file to the source of the file system command, such as to the attacker tool **204**.

(53) If the file system command is found **316** not to be a read command, such as in the case of a

write or a delete command, then the file system command is ignored and a simulated acknowledgment of the command is returned **316** to the source of the file system command.

(54) Referring to FIG. 3B, in some embodiments, the sensor module **108**, or a different sensor module **108** may execute the illustrated method **300b** with respect to commands other than file system commands. In some embodiments, a plurality of sensor modules **108** execute on the endpoint and each sensor module **108** may intercept a different types of command. Each sensor module will then execute the method **300a** or **300b** upon intercepting that type of command. For example, requests to modify, delete, or read information regarding processes executing on the endpoint, the registry of the endpoint, or an application programming interface (API) available to applications executing on the endpoint. The method **300b** may include receiving **320** a command and evaluating **322** whether the command references protected data or is a protected command. If not, then the command may be executed **324**. For example, the protected data **110b** may indicate which commands are restricted to sanctioned applications **110a** in addition to data or types of data that are protected. In some embodiments, only sanctioned applications are permitted to access any data or invoke execution of any command. Accordingly, step **322** may be omitted as a path to execution **324** of a command.

(55) The method **300b** may further include evaluating the source of the command of step **320** according to steps **308-312** in the same manner as for the method **300a**. If the conditions of steps **308-312** are met, then the command may be executed in the same form it was received. As for the method **300a**, all of steps **308-312** must have a positive result before the command is executed **324** and one or more additional tests may be required to be satisfied before the command is executed **324** without modification.

(56) If the steps **308-312** indicate that the source is a sanctioned application, then the method **300b** may include evaluating **326** whether the command is a request for information, such as a request for information regarding executing processes, the registry, available APIs, or other system information. If so, then deception data is returned **328** to the source of the command. As for other embodiments disclosed herein, the deception data that is returned may mimic the legitimate data that is requested but not correspond to actual system data.

(57) If the command is not a request for information, the method **300b** may include returning **330** a result that simulates successful execution of the command. As for the method **300a**, a delete command may indicate that data was successfully deleted, a modify command may indicate that the system data or process or operating parameter referenced in the command was modified as requested.

(58) An example of the use of the method **300a** may include the browser history for a browser. For example, the FIREFOX browser history may be stored at C:\Users\108 and instead receive a simulated browser history file including decoy data.

(59) In another example, in Windows, the “shadowcopy delete” command deletes the volume shadow copies. Malware often deletes this to accessing of backup volumes. Accordingly, this file may be listed as protected data **110b** and sensor module **108** will detect attempts to delete the volume shadow copies by non-sanctioned applications and prevent their execution. As noted above, acknowledgments of such commands may be returned indicating that the volume shadow copies were in fact deleted.

(60) In an example of the use of the method **30b**, an attacker, e.g. attacker tool **122**, tries to access the registry, which may be in a listing of protected data **110b**. For example, the attacker may attempt to read the registry to determine a registry key for one or more antivirus application (e.g., MCAFEE, SYMANTEC, or the like) is present in the registry. No legitimate application would

generally need to access the registry to determine whether an antivirus application is installed. In some embodiments, the sensor module **108** may intercept such attempts and return a result including a registry key for the antivirus tool, regardless of whether the antivirus application is installed. In many cases, this will cause the attacker tool to refrain from installing itself or otherwise attempting to perform malicious activities.

(61) In another example, the attacker, e.g. attacker tool **122**, seeks to determine whether the endpoint is a virtual machine (VM), such as by evaluating the registry to determine whether a hypervisor is installed and/or executing on the endpoint. The sensor module **108** intercepts these requests and returns an output indicating that the endpoint is executing a VM, e.g. indicate that the current operating environment in which the command was received is a VM or that a hypervisor is installed and/or executing on the endpoint. In another example, the attacker, e.g. attacker tool **122**, seeks to view a list of processes executing on the endpoint. In response, the sensor module **108** will return a “correct list of processes list” to the attacker.

(62) Referring to FIG. 4, the network environment **100** may further include an active directory server **402** in one or more of the domains **102a-102c**. The active directory server **402** may implement a directory service. A directory service functions as databases that map and store the names of network resources to their respective network addresses. Users referencing network objects need not remember the physical address of the object. The directory may store network resources and those resources may or may not have a name. The directory can also store user identifiers (very common), departments of user identifiers, access level of user identifiers, hosts accessible by user identifiers, the access level associated with each user identifier. The directory may further store organizational details, a network topology, an IT policy, and the like. Although Active Directory, the most popular directory server, depends on DNS very heavily, it is distinct therefrom and provides much more than a simple mapping between domain names and IP addresses. In particular, directory services may perform authentication for access to resources as well. LDAP (Lightweight Directory Access Protocol) is one of the popular methods available to access the data in directory services. LDAP also provides authentication and authorization to let user access resources in directory services.

(63) The directory service implemented by the active directory server **402** may provide authorization and access to key assets in corporate networks. Attackers may use various methods of exploitation to get unauthorized access to directory services. Once an attacker obtains access to a directory service, the attacker can easily log into key servers, databases etc. by impersonating credentials stored in directory services. The attacker may then exfiltrate data. For example, confidential data may be disclosed. In some instances, unauthorized modifications could be made (such as new account creation, access control modifications, document forgery, backup corruption, unauthorized financial transactions etc.) or data may be made unavailable (such as crypto malware, bank account takeover, bringing down or defacement of corporate web servers).

(64) The systems and methods disclosed herein incorporate database deception into directory services so that attackers cannot differentiate between real production assets with respect to fake assets. Once an attacker access fake credentials pointing to a BotSink **120**, the system prevents outbound access and simulates access to actual enterprise assets. This enables the identification and analysis of attackers for use in preventing data breaches.

(65) “Active Directory” (AD) is one of the directory services supported by WINDOWS operating systems. The AD domain controller is a server that provides authentication services within a domain whereby it provides access to computer resources. AD provides a database storing information about objects. Each object can be a user, computer, or a group of users. Although the systems and methods disclosed herein are described for use with an Active Directory system, they may be applied with equal effectiveness to any directory service.

(66) The systems and methods described herein returns simulated responses to requests to an active directory server **402**. These responses reference the BotSink **120** and may include credentials for

authenticating with respect to the BotSink **120**, such as a user account defined on the BotSink **120** or a service executing on the BotSink **120**. These responses lure an attacker to use the service or to attempt to use the user account. However, use of such services or accounts results in engagement of the attacker with the BotSink **120** and other resources described herein. This enables early detection of malicious intent and account misuse and diverts attention from real targets and resources.

(67) Referring to FIG. 5, in some embodiments, the sensor module **108** on an endpoint **106a-106g** may detect responses from an active directory server **402**. These responses may be the result of requests issued to the active directory server **402** by an application **202** or attacker tool **204** executing on that endpoint **106a-106g**. For example, the sensor module **108** may operate in conjunction with substituted or modified functions in an active directory API (application programming interface) in the operating system **200** of an endpoint **106a-106g**. The substituted or modified functions may route responses from the active directory server **402** to the sensor module **108** prior to return of the response to the application that requested it. The sensor module **108** may then determine whether to return the response to an application **202** to which the response is addressed without modification or with modification. In particular, if the response is addressed to an attacker tool **204** that is not a sanctioned application, the response may be modified such that any system, user account, or credential included in the response is replaced with corresponding references to the BotSink **120**, decoy computer on the BotSink **120**, a credential or user account for authenticating with the BotSink **120** (e.g., logging in, accessing a particular service, etc.).

(68) The manner in which the application to which the response is addressed is determined to be sanctioned or not may be according to the approach described in either of FIGS. 3A and 3B. In particular, the response from the active directory server **402** may be received on a port with respect to which an application is registered by the operating system **200**. Accordingly, the sensor module **108** may evaluate that application to determine whether it is sanctioned according to the approach of either of FIGS. 3A and 3B. If not, the response is modified at step **606** to replace references to addresses, actual user accounts, services, computer systems, or other production resources with corresponding references to the BotSink **120**, i.e. an address of the BotSink **120**, a decoy user account, decoy service, decoy virtual machine, or other decoy resource executing on the BotSink **120**.

(69) Referring to FIG. 6, the sensor module **108** may execute the illustrated method **600** with respect to responses from the active directory server **402**. The method **600** includes receiving **602** a response from the active directory server **402** by the sensor module **108**. The sensor module **108** evaluates **604** whether the application to which the response is addressed is sanctioned. For example, this may include evaluating an application listening to a port to which the response was addressed. Alternatively, the response may include some other identifier that is sufficient to identify the application (i.e. instance of an application executing on operating system **200** to which the response is addressed. As noted above, receiving the response may be performed by interfacing with an API programmed to interface with the active directory server **402** such that response are intercepted before they are returned to the requesting application. For example, the sensor module **108** may interface with substituted or modified versions of the Power Shell such that commands input to the Power Shell are intercepted and possibly modified by the sensor module **108**. Commands that may be intercepted may include commands such as ‘net group/domain “domain users”’ will return list of domain users present in Active Directory. Other example commands include ‘Get-ADComputer-Filter *’ will return all computer accounts in Active Directory, “nltest/dclist:domain name” will return domains in the network.

(70) The modified response is then returned **608** to the application to which it is addressed, i.e. the application that requested the information included in the response from the active directory server **402**. Where the application is found **604** to be sanctioned, the original response, or data from the response, is returned **608** to the source of the request without substituting references to the BotSink

120.

(71) Note that FIG. 6 illustrates the case where a response from the active directory service is received and modified. In other embodiments, a request from an application that is not sanctioned that is addressed to the active directory service is intercepted and simulated response is generated that references the BotSink **120**, i.e. a response that has an expected format corresponding to the request but with names of systems, addresses, credentials, services, etc. referencing such entities as implemented by the BotSink **120**.

(72) In some embodiments, the active directory server **402** may host decoy information describing a configuration of the BotSink **120**, such as decoy information describing decoy users accounts, addresses of decoy systems (virtual machines), decoy services, and decoy credentials for accessing any of these accounts, systems, and services. Accordingly, modifying a request by the sensor module may include modifying the request such that it is a request for decoy information on the active directory server (e.g., references the BotSink **120**) such that the response from the active directory server **402** includes only the decoy information rather than the information actually requested.

(73) In some embodiments, authenticated users of an endpoint **106a-106g** are added to a whitelist accessed by the sensor module **108** such that requests to the active directory server **402** and responses received are not substituted according to the method **600**.

(74) Referring to FIG. 7, in some embodiments, the sensor module **108** intercepts responses to requests to a domain controller **700**. In some embodiments, the function of the domain controller **700** is performed by a domain joined computer. For example, the sensor module **108** may monitor one or both of requests to a network API **702** of the operating system **200** to the domain controller **700** and responses to such requests.

(75) For example, referring to the method **800** of FIG. 8, the responses may be responses to requests for session data issued received **802** from a network API **702**. A response determined **804** to be to a non-sanctioned application may be modified at step **806** to refer to the BotSink **120** and returned **808** to the non-sanctioned application. In one scenario, an attacker tool **204** executing on an infected endpoint (e.g., endpoint **106a**) requests session data from the domain controller **700** or domain joined computer **700** using information learned via earlier active directory queries.

(76) Responses determined **804** to be in response to requests from sanctioned applications **202** may be returned **808** to them without replacing a reference to a computer system in the response with a reference to the BotSink **120**. The manner in which an application is determined **804** to be sanctioned may be performed in the same manner described above with respect to either of FIGS. 3A and 3B.

(77) FIGS. 9A and 9B illustrate how the methods **600** and **800** may be used to redirect and occupy an attacker. In an unprotected case shown in FIG. 9A, an attacker tool **204** on an infected endpoint **106a** uses the active directory API on the infected system to access an active directory database **900** through the active directory server **402**. Using this information, the attacker tool **204** uses the network API to request session data in order to identity of an endpoint **106b** logged in to the domain account for a domain to which infected endpoint **106a** belong. The attacker tool may infect the administrator endpoint **106b** and access the active directory database **900** and discover that the admin user account is referenced in domain data **904** defining another group, e.g. an administrator group. The endpoint **106b** may further store credentials enabling the endpoint **106b** to access a service hosted by server **106f**.

(78) Accordingly, the attacker tool **204** on the infected system **106a** may attempt to move laterally by requesting information from the active directory server **402** in order to discover the endpoint **106b** of the admin user account, infect it, and use it to access the service on server **106f** as an administrator.

(79) FIG. 9B illustrates an alternative view seen by an attacker tool **204** on the infected endpoint **106a** when the methods according to FIGS. 4 through 6 are implemented. The attacker tool **204**

requests information from the active directory server **402**. The responses to these requests are modified to reference decoy information, such as decoy active directory data **906** having a same format as the active directory database **900** but with references to computer systems and services being replaced with references to the BotSink **120** (e.g. virtual machines executing on the BotSink **120**) and services executing on the BotSink **120**.

(80) A response to a request to obtain the identity of an administrator of a domain to which the endpoint **106a** belongs using the network API is intercepted and modified to reference to a decoy domain **908**. A request for the identity of the administrator of the decoy domain **908** may include a reference to a user account **912** or virtual machine logged in to a user account **912** on the BotSink **120**. That user account **912** may be defined as part of an admin domain **910** and that user account **912** may include credentials (e.g., VM for which the user account **912** is logged in may store or reference credentials) that are sufficient to authenticate a user with respect to a service **914** executing on the BotSink **120**.

(81) Accordingly, the attacker tool **204** may attempt to move laterally as in the case of FIG. **9A** but only engage the BotSink **120**. The activities of the attacker tool **204** may be monitored and logged by the BotSink **120** in order to characterize the attacker tool **204** and alerts may be generated to alert and administrator to remove the attacker tool **204** from the infected endpoint **106a**. Engaging, monitoring, logging, characterizing, and generation of alerts may be performed according to any of the approaches described in the incorporated applications.

(82) FIG. **10** illustrates a method **1000** that may be implemented using the system shown in FIG. **9B**. An attacker tool **204** executing on an infected endpoint (e.g., endpoint **106a**) requests **1002** session data using a network API for the domain of the infected endpoint, the session data indicating computers (e.g., computers in the same domain) that are connected to the active directory server **402**. For example, a network API command for requesting such session data may be used. The sensor module **108** intercepts **1004** a response to this request and determines that the attacker tool **204** is a non-sanctioned application attempting to access restricted data (see FIG. **6**). In response, the sensor module **108** returns **1006** the session data with references to one or more computers in the session data replaced with one or more addresses assigned to the BotSink **120**. In the alternative, the sensor module **108** intercepts the request for session data and creates a decoy response referencing the BotSink **120** rather than modifying a response from the active directory server **402**.

(83) In particular, the IP address of the computer logged in to the administrator account of the domain may be replaced with an IP address assigned to the BotSink **120**. For example, the BotSink **120** may acquire IP addresses in various domains as described in the incorporated applications. Each address added to the session data may be assigned to a virtual machine (VM) executing on the BotSink **120**.

(84) The attacker tool **204** may then attempt to access **1008** the computer logged in to the administrator account, which is in fact the BotSink **120**. The BotSink **120** (e.g., the VM assigned the IP address represented as that of the administrator's computer at step **1006** ("the admin VM")) may then engage **1010** the attacker tool. The activities of the attacker tool **204** with respect to the BotSink **120** may be monitored and stored in order to characterize the attacker tool **204** as described in the incorporated applications.

(85) In particular, the attacker tool **204** may attempt to obtain credentials or identifiers of services cached or stored by the admin VM. In response, these credentials may be returned **1012** to the attacker tool. However, these credentials or identifiers of services may reference services implemented by the BotSink **120**, such as by other VMs being executed by the BotSink **120**. The attacker tool **204** receives the credentials or identifiers and attempts to access **1014** the services represented by them.

(86) In response, the BotSink **120** authenticates the attacker tool **204** and engages **1016** with the attacker tool **204** using the service, i.e. executes commands from the attacker tool in accordance

with the service. Actions of the attacker tool **204** may be monitored and used to characterize the attacker tool **204** as described in the incorporated applications. In addition or as an alternative, interaction with the BotSink **120** at steps **1010**, **1012**, and **1016** by the attacker tool may trigger an alert or remedial action such as removing the attacker tool **204** from the infected endpoint **106a** or isolating (disconnecting) the infected endpoint **106a** from a network.

(87) Note that in some embodiments, one of the services implemented on the BotSink **120** may be a decoy active directory service that implements an interface for responding to commands in the active directory API and is programmed to response to these commands with decoy data referencing decoy computers, user accounts, and services implemented on the BotSink **120**.

(88) Referring to FIG. **11**, in some implementations, intercepting of system calls according to the foregoing and subsequent embodiments may be implemented by substituting a reference to a detour function **1100** in the place of a target function **1102** in source function **1104** such that when the source function **1104** executes code that would have called the target function **1102**, the detour function **1100** is called instead. For example, this reference replacement may be made in dynamic link libraries (DLL) of the operating system on the endpoint **106a-106g** or other files of the operating system.

(89) The detour function **1100** may perform evaluations described herein to determine whether the call should be handled in cooperation with the BotSink **120** or should be passed on to the target function. The detour function **1100** may also perform actions described herein relating to substituting decoy information in a response to a call to the target function **1102**. Where it is determined that the call should be routed to the target function **110** this may be performed by way of a pass through function **1106** or “trampoline” function, which invokes the target function **1102** using arguments passed to the detour function **1100** by the source function **1104**. The target function **1102** may return its result to the detour function **1100**, which then returns the result to the source function **1104**. Alternatively, the result may be routed to the detour function **1100** by way of the pass through function **1106**. Where the function call is routed to the BotSink **120**, a result from the BotSink **120** may be routed to the source function **1104** by the detour function **1100**.

(90) The processing of system calls according to the methods disclosed herein may be preceded by modifying source functions making the system calls on the endpoint **106a-106g** to refer to one or more detour functions as described above.

(91) FIG. **12** illustrates a method **1200** for identifying whitelisted applications. For a period, e.g. 1 day, 2 days, 1 week, etc. following processing of source functions **1104** on the operating system of one or more endpoints **106a-106g** to reference the detour function(s) **1100**, a learning mode may be invoked **1202**. During this period the detour functions **1100** may operate in a learning mode in which all calls to a target function **1102** are intercepted **1204** and the detour function **1100** invokes **1210** the target function **1102**. I.e., a call to a target function **1102** will be passed to the target function **1102**. The detour function **1100** may report data regarding calls during the learning period, particularly identifiers of applications or processes that issue the call, e.g., a registry key, executable name, file system path to the executable, hash of the executable, or other identifier of the calling application or process. The detour function **1100** may make this report to the BotSink **120** for each system call or make a report of each unique identifier that made a call during the learning period. Note also that in some embodiments, not all system calls are intercepted, i.e. have references thereto replaced with detour function(s) **1100**. For example, either (a) only references to certain target functions **1102** are replaced with references to detour function(s) **1100** or (b) some functions **1100** simply pass a function call to a corresponding target function **1100** while others perform interception, inspection, and substitution functions as described herein.

(92) If the application or process that issued the intercepted call is not found **1206** to be in a list of sanctioned applications, it is added **1208** to the list, i.e., the unique identifier of the application or process is added to the list. If the application or process that issued the intercepted call is found **1206** to already be in a list of sanctioned applications, then step **1210** is performed without

repeating step **1208**.

(93) In some embodiments, each endpoint **106a-106g** creates its own list of sanctioned applications during the learning period and then sends these to the BotSink **120**, which then compiles a list of sanctioned applications that are then distributed back to the endpoints **106a-106g**. Alternatively, the endpoints **106a-106g** may coordinate with one another to share identifiers of sanctioned applications such that each endpoint **106a-106g** has the same list. In another approach, each endpoint **106a-106g** compiles its own list of sanctioned applications and uses it according to the methods described herein without obtaining identifiers of other applications identified by other endpoints **106a-106g**.

(94) Referring to FIG. **13**, the illustrated method **1300** may be executed on an endpoint **106a-106g**. For example, the method **1300** may be executed by a detour function **1104**, sensor module **108**, or other component executing on the endpoint **106a-106g** (hereinafter “the endpoint”).

(95) The method **1300** may include receiving **1302** a call to display network shares mounted to the endpoint. Network shares are file systems on other computers remote from the endpoint that are mounted to the file system of the endpoint and accessed through the file system of the endpoint. The call may be the “net use” command in the MICROSOFT operating system. Other function calls may include GetLogicalDrives(), GetVolumeInformation(). This call may be intercepted **1304** by virtue of a reference to it being a call to a detour function **1100** substituted for a target function corresponding to the call. Interception may also be performed by a sensor module **108** or according to any interception approach described herein.

(96) The detour function **1100** or other component performing the method **1300**, may evaluate **1306** whether the source of the call is a sanctioned application, e.g. a unique identifier of the source of the call is in the list of sanctioned applications. If so, the detour function **1100** invokes **1308** the target function **1102**, i.e. the “net use” function and returns a result of the function to the source of the call. If not, then the detour function **1100** returns **1310** a simulated result that includes a reference to a network share on the BotSink **120**. For example, the actual response to the “net use” function call. The simulated result may be obtained by the detour function **1100** issuing a call to the corresponding target function **1102**, i.e. the net use call, receiving the response, and replacing references to network shares with references to the BotSink **120**. In other approaches, the simulated result is generated without generating a call to the target function **1102**.

(97) For example, the actual result from the “net use” function may be:

(98) TABLE-US-00001 Status Local Remote Network OK Z: \\network426.example.com
Microsoft Windows Network

(99) The result returned to the source of the call may be:

(100) TABLE-US-00002 Status Local Remote Network OK Z: \\network521.example.com
Microsoft Windows Network

(101) In this example, “network426.example.com” may be an actual server providing a network share. “network521.example.com” may be the domain name assigned to the BotSink **120**.

(102) Attempts by the source of the call to access the network shares may result in the source of the call engaging **1312** the BotSink **120**. For example, the BotSink **120** may create a network share referenced by the result returned at step **1310**. Any attempt to access the share may be responded to by providing simulated files. Attempts to access the network share may be monitored by connection attempts on BotSink **120** to host network426.example.com or by monitoring the DNS queries to the host network426.example.com. Attempts to access the BotSink **120** through the references in the modified response may invoke generation of an alert or other action to reduce risk to the endpoint or other network resources, such as blocking access to the endpoint or other network resources by the source address or executable that is the source of the call.

(103) Referring to FIG. **14**, the illustrated method **1400** may be executed on an endpoint **106a-106g**. For example, the method **1400** may be executed by a detour function **1100**, sensor module **108**, or other component executing on the endpoint.

(104) The method **1400** may include receiving **1402** a call to list a remote network resource or request to list credentials managed by a credential manager, or a request to list of domain controllers accessible from the endpoint, or a request to enumerate remote network shares. This call may be intercepted **1304** by virtue of a reference to it being a reference to a detour function **1100** when the interception DLL was loaded into the application or process making the call. Interception may also be performed by a sensor module **108** or according to any interception approach described herein.

(105) The detour function **1100** or other component performing the method **1400**, may execute **1404** the call on the operating system of the endpoint, such as by invoking the corresponding target function **1102**. The detour function **1100** may intercept **1406** a response to the call from the operating system. If the source of the call is found **1408** to be a sanctioned application, the result from the operating system is returned **1410** to the source of the call. The manner in which the source is determined to be sanctioned may be according to the approach of any of the methods described hereinabove.

(106) If not, then the detour function **1100** returns **1412** a simulated result that includes a reference to a network share on the BotSink **120**. For example, response from step **1406** may be modified to include references to the BotSink **120** in the place of references to network resources.

(107) For example, the system call may be to an API (application programming interface) of a credential manager, such as that provided by the WINDOWS CREDENTIAL MANAGER API. For example, such a call may include “CredEnumerateA(), CredReadA(), or other function call defined in Advapi32.dll. The result from the operating system received at step **1406** may be: TERMSRV/network426.internalcorp.com, UserName rdpuser@internalcorp.com, Password: HASH1 The modified result returned to the source of the call may be TERMSRV/network521.internalcorp.com, UserName decoyuser@internalcorp.com, Password: HASH2 where network426.internalcorp.com is a server for which the credential manager has a credential and network521.internalcorp.com is a domain name assigned to the BotSink **120**. Note that the username and hash of the password are also altered in this example.

(108) In another example, the call intercepted at step **1402** is a call to list domain controllers. Such a call may include “WNetOpenEnumA()”, “WNetEnumResourceA()”, or other function call defined in Mpr.dll of the Win32 API. In another example, the call is “NetShareEnum(. . .)” in Netapi32.dll of the Win32 API.

(109) For example, the source of the call may issue the call “net view bantestado05.” The operating system may return the result:

(110) TABLE-US-00003 Shared resources at bantestad05 Share name Type Used as Comment
NETLOGON Disk Logon server share SYS VOL Disk Login server share

(111) In particular, the domain controllers may return NETLOGON and SYSVOL shares type. A result referencing these shares would then be modified to either (a) remove them to hide them from malware or (b) replace them with references to a decoy domain controller implemented on the BotSink **120**. This hinders an attacker from fingerprinting or finding the domain controllers of a network.

(112) In another example, the call received at step **1402** is to find network shares on a remote machine using a call such as WNetOpenEnumA(), WNetEnumResourceA() in the Mpr.dll of the Win32 API or NetShareEnum() in the Netapi32.dll of the Win32 API.

(113) For example, the result of a call such as “net view sx1.testsystem/all” may include:

(114) TABLE-US-00004 Shared resources at sx1.testsystem Share name Type Used as Comment
ADMIN\$ Disk Remote Admin SHARE_CIFS Disk C\$ Disk Default share IPC\$ IPC Remote IPC

(115) The entries of this result indicate network shares from a remote system. These network shares may be hidden from a non-sanctioned application according to the methods described herein. For example, for a non-sanctioned application, the entries may simply be deleted, returning a response at step **1410** that does not indicate the presence of any network shares. This enables the hiding of

these network shares from malware so that it cannot find these shares to exploit them or spread a ransomware-type attack. Alternatively, network shares on the BotSink **120** may be substituted in the result.

(116) In another example, the call received at step **1402** is a request to get information about network servers and/or computers. For example, the “NetServerEnum()” call in the Netapi32.dll of the Win32 APL

(117) For example, the input call may return “NETWORK.426” when executed by the operations, where NETWORK.426 is an identifier of an actual server on the network. The detour function **1100** may return NETWORK.521, which refers to the BotSink **120**.

(118) In another example, the call at step **1402** is a call to enumerate a registry of recently accessed servers or other computers. Such a call may include RegEnum Value() in the Advapi32.dll of the Win32 API, which enumerates a list of remote servers or computers recently accessed using a remote desktop connection.

(119) The result at step **1406** may read as follow:

(120) TABLE-US-00005 HKEY_CURRENT_USER\Software\Microsoft\ Terminal server Client\Default MRU0 REG_SZ NETWORK.426

(121) The modified result returned at step **1410** may read as follows:

(122) TABLE-US-00006 HKEY_CURRENT_USER\Software\Microsoft\ Terminal server Client\Default MRU0 REG_SZ NETWORK.521 where NETWORK.426 refers to a server in the network other than the BotSink **120** and NETWORK.521 refers to the BotSink **120**.

(123) As for other embodiments, attempts to access the network resources in the modified response will be received by the BotSink **120**, which may then engage **1414** the source of the call in order to detect and characterize it. Attempts to access the BotSink **120** through the references in the modified response may invoke generation of an alert or other action to reduce risk to the endpoint or other network resources, such as blocking access to the endpoint or other network resources by the source address or executable that is the source of the call.

(124) Referring to FIG. **15**, the illustrated method **1500** relates to a circumstance in which malware obtains a local administrative credential for a security account manager (SAM) database, such as from a local security authority subsystem service (LSAS) process. In this case, the endpoint receives **1502** a query to obtain information regarding a privileged group database. The privileged group may be referenced by the data obtained by the malware. The query may be a call to SamEnumerateGroupsInDomain(), SamGetMembersInGroup() in the Win32 API.

(125) If the query is found **1504** to be from a sanctioned application according to any of the approaches described herein, then the end point executes **1506** the query and returns **1508** a result of the query to the application, process, or remote computer system from which the query was received. If the query is not found **1504** to be from a sanctioned application, the endpoint provides **1510** decoy data to the source of the call, i.e. data including references to users, groups, or both users and groups that do not correspond to actual users and groups on the endpoint or in the production SAM database of the endpoint and network **100**. The decoy users and/or groups may reference decoy users and groups defined on the BotSink **120**, e.g., in a decoy SAM database on the BotSink **120**.

(126) If a request is found **1512** to have been received that references a decoy user or group returned at step **1510**, the method **1500** may include providing decoy information from decoy accounts corresponding to the decoy user or group referenced in the request. For example, this may be performed in response to receiving a SamGetMembersInGroup() call that references a decoy group provided at step **1510**. Steps **1512** and **1514** may be performed any number of times. In particular, decoy data may be generated in response to each request such that an attacker will be fed decoy data to occupy and deceive the attacker.

(127) Further attempts to access or exploit the user accounts may be directed to the BotSink **120** that may engage **1516** the attacker, such as by implementing the user accounts on the BotSink **120**

and providing access to services in the context of the user accounts on the BotSink **120**. These activities may be monitored and characterized. Engaging **1516** may invoke generation of an alert or other action to reduce risk to the endpoint or other network resources, such as blocking access to the endpoint or other network resources by the source of the call.

(128) Referring to FIG. **16**, in some embodiments an endpoint **106a-106g** may execute the illustrated method **1600**. A source may issue a call that references a DNS (domain name service) address that must be resolved to an IP (internet protocol address). Such a call may include `DNSQuery_A( )` from the `Dnsapi.dll` of the Win32 API. This call is received **1602** by the endpoint, such as by way of a detour function **1100**, which evaluates **1604** whether the call was received from a sanctioned application according to any of the approaches described herein. If so, the endpoint invokes **1606** performing of the access request. In some embodiments, prior to performing step **1606**, the endpoint resolves the DNS address to an IP address, such as by issuing a call with the DNS address to a DNS server and receiving a response including the IP address. The endpoint may then invoke performing of the access request with the IP address by the operating system of the endpoint, i.e. invoke the target function **1102** corresponding to the call. Alternatively, the arguments of the call, including the DNS address may be passed to the target function **1102** without first resolving the DNS address to an IP address. The target function **1102** may then execute with respect to the IP address. For example, if the functions **1104**, **1106** are a “ping” command, the target function **1102** will ping the BotSink **120** and return a result of that function, i.e. when and whether replies to ping messages were received, a delay between a ping message and a reply, and the IP address of the destination of the ping message.

(129) If the source of the call is not found **1604** to be a sanctioned application, the DNS address may be resolved **1608** to an address that does not actually correspond to the DNS address. Instead, the DNS address is resolved **1608** to one of one or more IP addresses assigned to the BotSink **120**. The access request may be performed **1610**, e.g. target function **1102** invoked, with the arguments from the call received at step **1602** except that the DNS address is replaced with the IP address from step **1608**. The result of the access request is then returned to the source of the access request. For example, where the access request is a ping command, the result of pinging the BotSink **120** may be returned to the source. As for other embodiments disclosed herein, attempts to communicate using the information returned may result in engaging **1612** the attacker as described above.

(130) FIG. **17** is a block diagram illustrating an example computing device **1700** which can be used to implement the system and methods disclosed herein. The endpoints **106a-106g**, management server **112**, BotSink **120**, attacker system **122**, and active directory server **402** may also have some or all of the attributes of the computing device **1700**. In some embodiments, a cluster of computing devices interconnected by a network may be used to implement any one or more components of the invention.

(131) Computing device **1700** may be used to perform various procedures, such as those discussed herein. Computing device **1700** can function as a server, a client, or any other computing entity. Computing device can perform various monitoring functions as discussed herein, and can execute one or more application programs, such as the application programs described herein. Computing device **1700** can be any of a wide variety of computing devices, such as a desktop computer, a notebook computer, a server computer, a handheld computer, tablet computer and the like.

(132) Computing device **1700** includes one or more processor(s) **1702**, one or more memory device(s) **1704**, one or more interface(s) **1706**, one or more mass storage device(s) **1708**, one or more Input/Output (I/O) device(s) **1710**, and a display device **1730** all of which are coupled to a bus **1712**. Processor(s) **1702** include one or more processors or controllers that execute instructions stored in memory device(s) **1704** and/or mass storage device(s) **1708**. Processor(s) **1702** may also include various types of computer-readable media, such as cache memory.

(133) Memory device(s) **1704** include various computer-readable media, such as volatile memory

(e.g., random access memory (RAM) **1714**) and/or nonvolatile memory (e.g., read-only memory (ROM) **1716**). Memory device(s) **1704** may also include rewritable ROM, such as Flash memory. (134) Mass storage device(s) **1708** include various computer readable media, such as magnetic tapes, magnetic disks, optical disks, solid-state memory (e.g., Flash memory), and so forth. As shown in FIG. **17**, a particular mass storage device is a hard disk drive **1724**. Various drives may also be included in mass storage device(s) **1708** to enable reading from and/or writing to the various computer readable media. Mass storage device(s) **1708** include removable media **1726** and/or non-removable media.

(135) I/O device(s) **1710** include various devices that allow data and/or other information to be input to or retrieved from computing device **1700**. Example I/O device(s) **1710** include cursor control devices, keyboards, keypads, microphones, monitors or other display devices, speakers, printers, network interface cards, modems, lenses, CCDs or other image capture devices, and the like.

(136) Display device **1730** includes any type of device capable of displaying information to one or more users of computing device **1700**. Examples of display device **1730** include a monitor, display terminal, video projection device, and the like.

(137) Interface(s) **1706** include various interfaces that allow computing device **1700** to interact with other systems, devices, or computing environments. Example interface(s) **1706** include any number of different network interfaces **1720**, such as interfaces to local area networks (LANs), wide area networks (WANs), wireless networks, and the Internet. Other interface(s) include user interface **1718** and peripheral device interface **1722**. The interface(s) **1706** may also include one or more user interface elements **1718**. The interface(s) **1706** may also include one or more peripheral interfaces such as interfaces for printers, pointing devices (mice, track pad, etc.), keyboards, and the like.

(138) Bus **1712** allows processor(s) **1702**, memory device(s) **1704**, interface(s) **1706**, mass storage device(s) **1708**, and I/O device(s) **1710** to communicate with one another, as well as other devices or components coupled to bus **1712**. Bus **1712** represents one or more of several types of bus structures, such as a system bus, PCI bus, IEEE 1394 bus, USB bus, and so forth.

(139) For purposes of illustration, programs and other executable program components are shown herein as discrete blocks, although it is understood that such programs and components may reside at various times in different storage components of computing device **1700**, and are executed by processor(s) **1702**. Alternatively, the systems and procedures described herein can be implemented in hardware, or a combination of hardware, software, and/or firmware. For example, one or more application specific integrated circuits (ASICs) can be programmed to carry out one or more of the systems and procedures described herein.

Claims

1. A method performed by a computer system, the method comprising: configuring, by a computer system, a system call to refer to a function; intercepting a first response from a directory service to a source on the computer system before the first response is returned to the source, wherein the first response is a response to a request for network data received by the directory service, wherein the request for network data originates from the source on the computer system, wherein the request includes the system call; determining, by the computer system via the function, that the source on the computer system is not sanctioned by comparing the source to a list of sanctioned applications; in response to determining that the source is not sanctioned, generating, via the function, a simulated response comprising deception data and information regarding a decoy server; and returning the simulated response to the source on the computer system.
2. The method of claim 1, wherein an agent executing on the computer system intercepts the first response before it is returned to the source.

3. The method of claim 1, wherein the method is performed by an agent software executing on the computer system.
4. The method of claim 3, further comprising: substituting or modifying the function in a directory service application programming interface (API) to route the first response from the directory service to the agent software before the first response is returned to the source.
5. The method of claim 1, wherein determining that the source is not sanctioned comprises determining that the source is not in a list of sanctioned applications.
6. The method of claim 1, wherein the first response is intercepted using a substituted or modified function in a directory service application programming interface (API).
7. The method of claim 1, wherein determining that the source is not sanctioned comprises evaluating an application listening to a port to which the first response is addressed.
8. A computer system comprising: one or more processing devices; and one or more memory devices operably coupled to the one or more processing devices, the one or more memory devices storing executable code that, when executed by the one or more processing devices, causes the one or more processing devices to: configure, by a computer system, a system call to refer to a function; intercept a first response from a directory service to a source on the computer system before the first response is returned to the source, wherein the first response is a response to a request for network data received by the directory service, wherein the request for network data originates from the source on the computer system, wherein the request includes the system call; determine, by the computer system via the function, that the source on the computer system is not sanctioned by comparing the source to a list of sanctioned applications; in response to determining that the source is not sanctioned, generate, via the function, a simulated response comprising deception data and information regarding a decoy server; and return the simulated response to the source on the computer system.
9. The system of claim 8, wherein an agent software executing on the computer system intercepts the first response before it is returned to the source.
10. The system of claim 9, wherein the executable code, when executed by the one or more processing devices, further causes the one or more processing devices to: substitute or modify the function in a directory service application programming interface (API) to route the first response from the directory service to the agent software before the first response is returned to the source.
11. The system of claim 8, wherein determining that the source is not sanctioned comprises determining that the source is not in a list of sanctioned applications.
12. The system of claim 8, wherein the first response is intercepted using a substituted or modified function in a directory service application programming interface (API).
13. The system of claim 8, wherein determining that the source is not sanctioned comprises evaluating an application listening to a port to which the first response is addressed.
14. A non-transient computer readable medium containing program instructions for causing a computer system to perform the steps of: configuring, by a computer system, a system call to refer to a function; intercepting a first response from a directory service to a source on the computer system before the first response is returned to the source, wherein the first response is a response to a request for network data received by the directory service, wherein the request for network data originates from the source on the computer system, wherein the request includes the system call; determining, by the computer system via the function, that the source on the computer system is not sanctioned by comparing the source to a list of sanctioned applications; in response to determining that the source is not sanctioned, generating, via the function, a simulated response comprising deception data and information regarding a decoy server; and returning the simulated response to the source on the computer system.
15. The non-transient computer readable medium of claim 14, wherein an agent executing on the computer intercepts the first response before it is returned to the source.
16. The non-transient computer readable medium of claim 14, wherein the program instructions are

for an agent software, and wherein the steps are performed by the agent software executing on the computer system.

17. The non-transient computer readable medium of claim 16, wherein the program instructions further cause the computer system to perform the step of: substituting or modifying the function in a directory service application programming interface (API) to route the first response from the directory service to the agent software before the first response is returned to the source.

18. The non-transient computer readable medium of claim 14, wherein determining that the source is not sanctioned comprises determining that the source is not in a list of sanctioned applications.

19. The non-transient computer readable medium of claim 14, wherein the first response is intercepted using a substituted or modified function in a directory service application programming interface (API).

20. The non-transient computer readable medium of claim 14, wherein determining that the source is not sanctioned comprises evaluating an application listening to a port to which the first response is addressed.
